

Optimizing data permutations in structured loads/stores translation and SIMD register mapping for a cross-ISA dynamic binary translator

Sheng-Yu Fu^{a,*}, Ding-Yong Hong^b, Yu-Ping Liu^a, Jan-Jan Wu^b, Wei-Chung Hsu^a

^a National Taiwan University, Taiwan

^b Academia Sinica, Taiwan

ARTICLE INFO

Keywords:

SIMD
Dynamic binary translation
Structured load/store
SIMD register mapping

ABSTRACT

More and more modern processors have been supporting non-contiguous SIMD data accesses. However, translating such instructions has been overlooked in the Dynamic Binary Translation (DBT) area. For example, in the popular QEMU dynamic binary translator, guest memory instructions with strides are emulated by a sequence of scalar instructions, leaving a significant room for performance improvement when the SIMD instructions are available on the host machines. Structured loads/stores, such as VLDn/VSTn in ARM NEON, are one type of strided SIMD data access instructions. They are widely used in signal processing, multimedia, mathematical, and 2D matrix transposition applications. Efficient translation of such structured loads/stores is a critical issue when migrating ARM executables to other ISAs. However, it is quite challenging since not only the translation of structured loads/stores is not trivial, but also the mapping of SIMD registers between guest and host is complicated. This paper presents the design of translating structured loads/stores in DBT, including target code generation, efficient SIMD register mapping, and optimizations for reducing data permutations. Our proposed register mapping mechanisms and optimizations are not limited to handle structured loads/stores, they can be extended to deal with normal SIMD instructions. This paper evaluates how different factors affect the translation performance and code size. These factors include guest SIMD register length, strides, and use cases for structured loads. On a set of OpenCV benchmarks, our QEMU-based system has achieved a maximum speedup of 5.03x, with an average improvement of 2.87x. On a set of BLAS benchmarks, our system has obtained a maximum speedup of 2.22x and an average improvement of 1.78x.

1. Introduction

Modern microprocessors, from high-end servers to mobile devices, all exploit data parallel computing through SIMD (Single-Instruction Multiple Data) execution. A typical SIMD operation can either compute multiple data elements or transfer them from/to the memory in parallel. Vector supercomputers in early 90's had word-interleaved memory banks and thus used vector load/store instructions for strided access and gather/scatter operations. Microprocessors are often designed around cache hierarchies. Their main memory systems are normally cache line interleaved, so supporting strided load or gather/scatter may likely encounter more memory traffic and cache misses. As a result, SIMD loads/stores in many microprocessors are often based on contiguous memory operations [12]. However, many applications, including multimedia, digital signal processing, and scientific computing, rely on algorithms or data structures that require non-contiguous or small-stride memory accesses [25,32]. For such applications of non-contiguous memory access patterns, they are usually not vectorized. Many prior compiler researches have investigated different interleaved

access patterns. They found that certain interleaved access patterns can be vectorized with contiguous SIMD loads/stores together with data reorganization operations such as shuffling and permutation to significantly boost performance [6,15,26,27].

Processor vendors are also aware of the importance of supporting non-contiguous data accesses and have enhanced their SIMD instruction set. For example, Intel x86-AVX2 includes gather instructions that can gather multiple data elements from arbitrary memory locations. In contrast, ARM NEON includes structured loads/stores to support some stride-access patterns, which transfer chunks of structured data in the memory and then de-interleaves/interleaves structured data elements to/from multiple SIMD registers based on the specified stride. Such hardware supports can reduce the data reorganization overhead. With the advancement of aforementioned software and hardware approaches, more applications of non-contiguous memory accesses are expected to be fully vectorized. However, the issue of how to translate non-contiguous SIMD memory instructions has not been addressed in the context of cross-ISA Dynamic Binary Translation (DBT).

* Corresponding author.

E-mail address: d03922013@csie.ntu.edu.tw (S.-Y. Fu).

Cross-ISA DBT is a common technique to execute an application binary of one guest instruction set architecture (ISA) on a host with a different ISA, providing the powerful functionality for cross-ISA application migration and virtual platforms building. For example, many Android applications are ARM based. Based on the DBT technique, the Android emulator can emulate such ARM-based applications in the Android environment on x86-based platforms. Recently, several works have proposed approaches for guest SIMD instruction translation in cross-ISA DBT [4,8,24]. Based on their approaches, the guest SIMD instructions can be translated to host SIMD instructions. However, these works do not address the non-contiguous load/store translation. They emulate guest memory instructions with strides by a sequence of scalar instructions, leaving a significant room for performance improvement when the host machines have SIMD instructions available.

Since different ISAs provide various support for non-contiguous loads/stores, it would be interesting to understand how a dynamic binary translator could efficiently transform one ISA form of non-contiguous loads/stores to another ISA form. For example, ARM NEON supports structured loads (VLDr) while x86-AVX2 supports gather. When migrating ARM executables to x86 machines, how can such structured loads from ARM NEON be efficiently translated into various combinations including: a) all scalar instructions if the host does not have SIMD instructions, b) regular contiguous SIMD loads with permutations if the host only supports contiguous SIMD memory operations, or c) SIMD gathers if the host has gather/scatter support.

Furthermore, the emulation of structured loads and stores is typically permutation and memory bound. When emulating guest programs, dynamic binary translators need to conduct register mapping, a process to allocate host registers for emulating guest architectural states. Improper mapping can incur excessive memory operations or high data reorganization overhead. A simple approach is to emulate guest SIMD registers in memory and generate load/store instructions to access them on the host. However, this naïve translation generates excessive load and store operations, and thus drastically degrades the performance. An improved approach is to map guest SIMD registers to host SIMD registers. However, this approach poses a challenge to register mapping in DBT because the guest and host could have different SIMD register structures. For example, ARMv7 provides name aliases for accessing any subsets of a SIMD register (e.g., a quadword register Q1 can be accessed by two double-word registers, D2 and D3) where some other ISAs disallow aliased accessing. Therefore, how to select an efficient mapping between different SIMD register layouts is important in maintaining the SIMD execution performance.

In this paper, we look at those design issues and implement the proposed schemes in a DBT framework that consists of QEMU frontend and LLVM backend. Four structured load/store translation approaches are proposed for hosts supporting different kinds of non-contiguous memory operations. Moreover, to further reduce data permutations, we design a heuristic-based register mapping algorithm together with the optimization of *GuestMemoryFusion*. The major contributions of this paper are:

1. We define a set of generalized structured load/store IRs in QEMU TCG IR, which facilitates the translation of SIMD structured loads/stores. We propose four structured load/store translation schemes for hosts supporting different kind of non-contiguous memory operations (Section 3).
2. We design a heuristic-based register mapping algorithm, including a cost model for evaluating the cost of register mapping configurations (Section 4).
3. We propose the optimization, *GuestMemoryFusion*, to reduce data permutations as well as L1 data cache access. (Section 5).
4. We evaluate our translation strategies with the translations from two guest ISAs, ARMv7 and ARMv8, to two host ISAs, x86-SSE4 and x86-AVX2, respectively. On a set of OpenCV benchmarks, our system can achieve a maximum speedup of 5.03x, with an average improvement of 2.87x (Section 6). We also evaluate our algorithms against arbitrary

strides, register length, and structured load use cases to discuss the pros and cons and the general applicability of our algorithms (Section 7).

This article extends our previous work, which focuses on the design of structured loads/stores translation, and optimizations for complex multiplication and structure field reordering [7]. The extensions include the condensed gather/scatter translation algorithm, the *GuestMemoryFusion* optimization, the validation for the cost model of register mapping algorithms, and the new experiments for evaluating the pros and cons of the proposed translation algorithms. The rest of the paper is organized as follows. Section 2 gives the background. Section 3 describes four translation approaches of structured loads/stores. Section 4 presents the SIMD register mapping algorithms. Section 5 presents the optimizations to further reduce data reorganizations. Sections 6–8 evaluate our approaches. Section 9 discusses the translation from ISAs that do not support structured loads/stores to ISAs supporting structured loads/stores. Section 10 covers related work and Section 11 concludes.

2. Background

In this section, we review register structures and memory instructions in different SIMD architectures. We then introduce the generalized form of structured load/store instructions.

2.1. SIMD register layout and challenges in DBT register mapping

SIMD extensions have been adopted by many microprocessors such as ARM and x86, however, their SIMD register layouts are rather different. For example, ARMv7 provides name aliases for software to access any subsets of a SIMD register, while other SIMD architectures stay away from such full aliasing, as illustrated in Fig. 1. Different SIMD register structures pose challenges to register mapping in DBT. A simple approach for a dynamic binary translator to maintain guest architecture states is to map all guest registers to memory, and generate load/store instructions to access them on the host. However, this naïve translation yields excessive loads and stores. One efficient implementation is to keep all or part of the guest register values in the host registers. This is relatively simple if the host and guest registers have similar addressing ability. For example, ARMv8 has 32 64-bit general-purpose registers (*X* register), and each of them contains a lower 32-bit sub-register (*W* register). Moreover, the higher 32-bit sub-register of *X* register is not addressable. This is similar to the *RAX* and *EAX* register on x86-64. However, SIMD register mapping is non-trivial because the guest and host may have very different SIMD structures. For instance, ARMv7 NEON registers are full-aliased, allowing the use of *S/D* registers to address any 32-bit/64-bit lane of the *Q0* registers. Unlike ARMv7 NEON, x86 AVX2 only allows partial aliasing. Only the lower part of *YMM* registers can be addressed by the *XMM* registers. Hence, a good SIMD register mapper should consider not only the variances between guest and host register structures (e.g., register length, number, and addressing ability), but also how registers are referenced in the guest binary.

2.2. SIMD structured loads/stores and the vectorization

Non-contiguous SIMD memory operations can be classified into three categories: strided, gather/scatter, and structured operations. A SIMD strided load or store instruction transfers multiple data elements whose memory locations are separated by a constant stride. (SIMD load/stores with contiguous memory locations are often called stride-one or unit-stride references.) In contrast, a SIMD gather or scatter instruction can transfer multiple data elements from/to arbitrary memory locations. Such a non-contiguous memory operation frequently requires accesses to multiple cache lines, resulting in additional memory traffic as well as potential cache pollution.

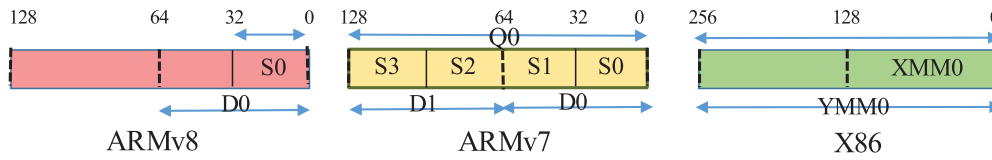


Fig. 1. SIMD register layouts in x86, ARMv7 and ARMv8.

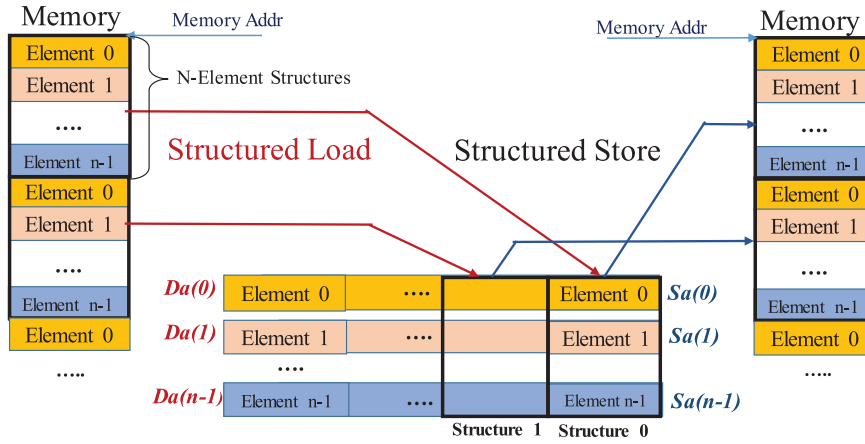


Fig. 2. The structured load and store operations.

```

1 for(i=0; i<size; i++){
2   a[i].re= b[i].re*c[i].re - b[i].im*c[i].im ;
3   a[i].im= b[i].re*c[i].im + b[i].im*c[i].re ;
4 }

```

Fig. 3. Complex number multiplication (Fully-used).

A variation of SIMD strided memory operations is structured memory operations, which are implemented in the ARM NEON architecture. A SIMD structured load/store instruction transfers chunks of structured data in the memory and then de-interleaves/interleaves structured data elements to/from multiple SIMD registers based on the specified stride. The semantics of a general structured load/store instruction can be formulated as follows:

- *StructuredLoad*(Stride, $D_{a(0)}, \dots, D_{a(N-1)}, \text{ElementSize}, \text{MemoryAddr}$),
- *StructuredStore*(Stride, $S_{a(0)}, \dots, S_{a(N-1)}, \text{ElementSize}, \text{MemoryAddr}$),

where $N = \text{Stride}$, $D_{a(i)}$ and $S_{a(i)}$, $0 \leq i < N$ respectively are the destination and source SIMD registers. Fig. 2 illustrates the flows of structured load and store operations. In Fig. 2, each structure consists of N elements, and multiple structures are loaded/stored into/from N registers. The first elements of all structured data in the memory are separated by a distance of Stride, as are the other structure elements. For a structured load operation, the first elements from all structured data are collected and packed into the first register $D_{a(0)}$, all the second elements are collected and packed into the second register $D_{a(1)}$, and so on—a de-interleaving operation. A structured store operation performs similar operations but in a reverse direction. We use the ARMv7 structured load instruction, `vld3.i8 {d0, d2, d4}, [r0]`, as an example: each structure contains three element of chars and eight consecutive structured data pointed by `r0` are de-interleaved into three registers, `d0`, `d2`, and `d4`. The general form, *StructuredLoad*(3, {`d0 d2 d4`}, `i8, r0`), can formulate such a `vld3` instruction.

Structured loads transfer and de-interleave all structure elements to SIMD registers. The naïve use cases for structured loads are applications accessing all structured elements. For example, complex number multiplication accesses both real and imagination parts, as shown in Fig. 3. Recent compilers such as GCC-7 have been able to vectorize patterns accessing partial structured elements, e.g., brightening up the red channel and moving the 3D objects along X direction, by leveraging structured

```

1 for(i=0; i<size; i++){
2   NewObj.p[i].x =
3     OldObj.p[i].x + 1;
4 }

```

Fig. 4. Move 3D objects along X direction (Partially-used).

load instructions. Fig. 4 illustrates the example moving 3D objects along X direction which only uses X components. Based on these vectorization techniques, structured loads/stores are expected to be more and more widespread and make more applications vectorized.

3. Translating SIMD structured loads/stores

The translation of SIMD structured loads/stores in a binary translator involves three parts: (1) translating the guest structured loads/stores into vector IRs; (2) selecting a register mapping where the guest SIMD registers can mostly reside in the host SIMD registers so as to minimize memory and data reorganization operations; and (3) translating vector IRs into host SIMD instructions. In this section, we describe the approaches for translating guest structured loads/stores into vector IRs.

3.1. Translation approach

A naïve approach of translating SIMD structured loads/stores is to emulate a SIMD structured load/store instruction by a sequence of *scalar* instructions. This approach is designed in the popular dynamic binary translator, QEMU, where no SIMD operations are generated even if the host architecture does support SIMD instructions. Fig. 5(a) shows a translation example of a structured load with QEMU. One structure data element is loaded by one scalar load at a time and then inserted into the corresponding location of the emulated guest registers. Such a translation is simple and portable since it only involves scalar operations. However, this would leave a lot of untapped performance on host machines equipped with SIMD computing capabilities.

To exploit the SIMD capabilities on the host, our translator needs to consider different numbers of SIMD lanes, data reorganization utilities, and memory accessing capabilities (e.g., contiguous SIMD loads/stores, strided loads/stores, and gather/scatter operations) provided by the host

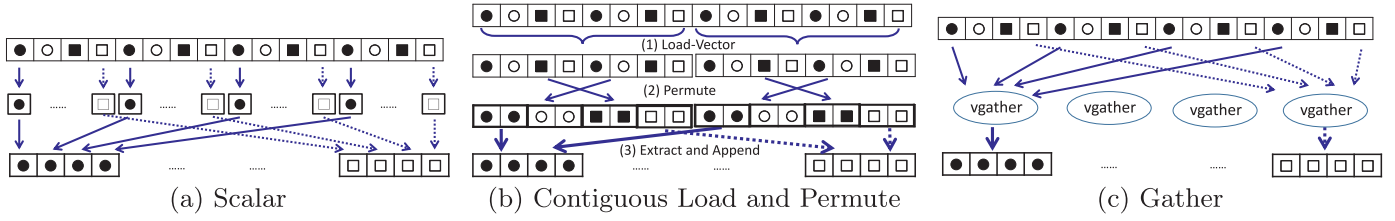


Fig. 5. Translation approaches of the SIMD structured load operation.

SIMD architecture. This enhancement work poses considerable challenges.

For hosts supporting only contiguous SIMD loads and stores, we propose a translation scheme using contiguous SIMD memory operations plus permutation operations. This approach is based on the observation that most modern processors support contiguous SIMD memory loads/stores and permute operations of primitive data types. As shown in Fig. 5(b), the structured data are loaded as contiguous chunks to host SIMD registers. Next, the structured data elements are redistributed to their corresponding locations among the SIMD registers using data reorganization instructions such as pack, unpack, or permute instructions.

For hosts that support SIMD gather/scatter operations, the non-contiguous data elements distributed among different structures can be directly transferred using gather/scatter instructions. Fig. 5(c) shows an example using gather instructions to implement a structured load operation where each list of structure data elements is collected by one gather operation.

Intuitively, just translating guest structured loads/stores into host gathers/scatters seems like a general solution. However, several problems limit the use of gather/scatter operations. First, many modern microprocessors, such as MIPS, PowerPC, and so on, do not support gather/scatter operations, and only gather operations are supported in the x86-AVX2 ISA. Second, not all primitive types are supported in gather/scatter operations. For example, the x86 processors can only gather/scatter 32-bit or 64-bit data elements. As a result, structured load/store operations of other data types (e.g., transforming color space of 8-bit pixels) cannot be mapped directly into gather/scatter instructions. For these reasons, we must provide a more general translation solution based on using contiguous SIMD memory operations together with permutation operations.

- A vector type is represented as $\langle \text{number of elements} \times \text{element type} \rangle$. For instance, $\langle 32 \times i8 \rangle$ represents for the vector type of 32 8-bit integer values.
- Instruction *vload* loads contiguous data of type $\langle \# \text{ loaded element}, \text{element type} \rangle$ to a SIMD register from the memory location *source_addr*.
- Instruction *vstore* stores contiguous data of type $\langle \# \text{ stored element}, \text{element type} \rangle$ from a SIMD register to the memory location *destination_addr*.
- Instruction *permute* returns the vector value by permuting a SIMD register based on the given index. For instance, *permute*($\langle a, b, c, d \rangle, \langle 0, 2, 2, 3 \rangle$) returns $\langle a, c, c, d \rangle$.
- Instruction *append* returns the vector value by appending the latter to the former. For instance, *append*($\langle a, a \rangle, \langle b, b, c \rangle$) returns $\langle a, a, b, b, c \rangle$.
- Instruction *extract* returns sub-vector between [begin, end) index interval. For instance, *extract*($\langle a, b, c, d, e \rangle, 1, 3$) returns $\langle b, c \rangle$.
- Instruction *vgather* loads multiple data elements from the memory locations indexed by (*source_addr* + *index*[*i*]) and packs them into a SIMD register.
- Instruction *vscatter* stores data elements from the SIMD register *source_vector* to the memory locations indexed by (*destination_addr* + *index*[*i*]).

3.2. Translation of SIMD structured loads/stores

We design the proposed translation schemes of SIMD structured loads/stores in the IR layer. Three categories of pseudo instructions are introduced in our dynamic binary translator IR: continuous load/store, permute/append/extract, and gather/scatter. Table 1 lists the semantics of the pseudo instructions.

In the following subsections, we describe how to translate SIMD structured loads by using these pseudo instructions for the example in Fig. 5. We use SIMD structured load operations here for the description; the implementation of the structured store operations is similar but with a reverse direction, as shown in Fig. 6.

3.2.1. Translation using contiguous loads/stores and permutation

Algorithm 1 shows the pseudo code used to translate a SIMD structured load operation using contiguous SIMD loads and permutation. Here we use Fig. 5(b) as an example for the descriptions of the algorithm—assuming that stride of accessing is 4; the element type is 32-bit

Algorithm 1: Contiguous L/S and Permutation.

```

input : Stride, Destination registers  $D = \{d_0, \dots, d_{s-1}\}$ ,
Number of lanes to access per guest register—GuestLane,
Number of lanes to access per host register—HostLane,
Host register width—HostWidth
1 Function Translate-Load(Stride, D, Ptr, GuestLane, HostWidth)
   /* NumAS: # of structures accommodated in a host SIMD register */
2   NumAS  $\leftarrow \min\{ \lfloor \text{HostWidth} / (\text{Stride} \times \text{ElementSz}) \rfloor, \text{GuestLane} \}$ 
3   if NumAS > 1 then
   /* Host can accommodate multiple structure. */
4     NumSplit  $\leftarrow \text{GuestLane} / \text{RoundDownToPowerOf2}(\text{NumAS})$ 
5     LoadLane  $\leftarrow (\text{Stride} \times \text{GuestLane}) / \text{NumSplit}$ 
6     SplitLane  $\leftarrow \text{GuestLane} / \text{NumSplit}$ 
7      $k \leftarrow \text{Permute-Mask}(\text{Stride}, \text{SplitLane})$ 
8     for  $i \leftarrow 0$  to NumSplit - 1 do
9        $v \leftarrow \text{Load-Vector}(\text{Ptr}, \text{ElementTy}, \text{LoadLane})$ 
10       $v \leftarrow \text{Permute-Vector}(v, k)$ 
11      foreach  $d_j \in D$  do
12         $d_j.\text{append}(\text{Extract}(v, \text{SplitLane} \times j, \text{SplitLane} \times (j + 1)))$ 
13      end
14       $\text{Ptr} \leftarrow \text{Ptr} + \text{LoadLane} \times \text{ElementSz}$ 
15    end
16  end
17  else
18    NumFullLoad  $\leftarrow \lfloor (\text{Stride} \times \text{GuestLane}) / \text{HostLane} \rfloor$ 
19    RemLane  $\leftarrow (\text{Stride} \times \text{GuestLane}) \bmod \text{HostLane}$ 
20    for  $i \leftarrow 0$  to NumFullLoad - 1 do
21       $v_i \leftarrow \text{Load-Vector}(\text{Ptr}, \text{ElementTy}, \text{HostLane})$ 
22       $\text{Ptr} \leftarrow \text{Ptr} + \text{HostWidth}$ 
23    end
24    if RemLane then
25       $v_{\text{NumFullLoad}} \leftarrow \text{Load-Vector}(\text{Ptr}, \text{ElementTy}, \text{RemLane})$ 
26    end
27    foreach  $d_j \in D$  do
28      for  $i \leftarrow 0$  to GuestLane - 1 do
29         $\text{idx} \leftarrow \lfloor (\text{Stride} \times i + j) / \text{HostLane} \rfloor$ 
30         $\text{offset} \leftarrow (\text{Stride} \times i + j) \bmod \text{HostLane}$ 
31         $d_j.\text{append}(\text{Extract}(v_{\text{idx}}, \text{offset}, \text{offset} + 1))$ 
32      end
33    end
34  end

```

Table 1
Semantics of the pseudo instructions.

Instruction	Arguments	Instruction	Arguments
vload	(source addr, element type, # element)	append	(source1, source2)
vstore	(destination addr, element type, # element)	extract	(source vector, begin index, end index)
permute	(source vector, index)	vgather	(source addr, index, element type)
		vscatter	(destination addr, index, source vector)

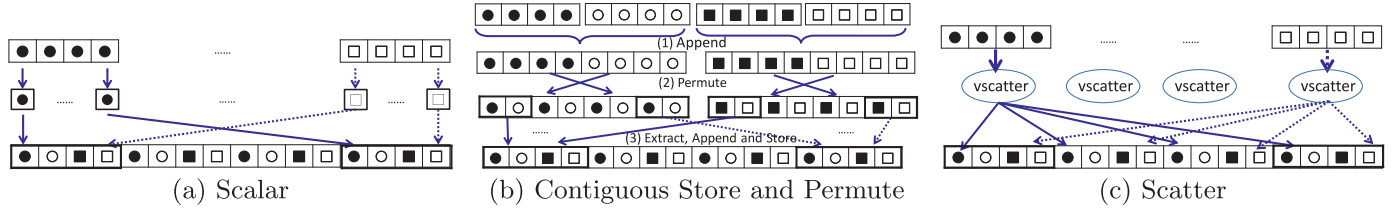


Fig. 6. Translation approaches of the SIMD structured store operation.

integer; the width of host SIMD register is 256-bit; the number of guest SIMD lanes is 4 (e.g., translating an ARM NEON vld4.i32 instruction), and the number of host SIMD lanes is 8.

The algorithm consists of the following steps: (1) We first determine whether a host SIMD register can accommodate multiple structured data or not (line 2). (2) If a host SIMD register can store multiple structures (line 3), we calculate how to split the structured data and the number of contiguous loads required to transfer the data (line 4–6). Because the *GuestLane* is power of two, we round down *NumAs* to power of two to make sure *NumSplit* is an integer (line 4). (3) Based on the *Stride* and *NumSplit*, we compute the permutation mask, *k* (line 7, Algorithm 2). For the example in Fig. 5(b), the index *k* is $\langle 0, 4, 1, 5, 2, 6, 3, 7 \rangle$.

Algorithm 2: Permutation Mask.

```

input : Stride,
        Number of lane to access per register–NumLane
output: Permutation mask used for reordering–k
1 Function Permute-Mask(Stride, NumLane)
2    $k \leftarrow \langle \rangle$ 
3   for  $i \leftarrow 0$  to  $\text{Stride} - 1$  do
4     for  $j \leftarrow 0$  to  $\text{NumLane} - 1$  do
5        $k.append(\text{Stride} \times j + i)$ 
6     end
7   end
8   return  $k$ 

```

(4) Load-Vector then generates *vload* to fetch contiguous memory of multiple structures into a host SIMD register *v* (line 9). (5) After multiple structures are loaded, the permutation index *k* is used to move structured data elements to corresponding locations in the SIMD register *v*, where elements belonging to the same destination register are placed together (e.g., adjacent lanes) (line 10). (6) Adjacent lanes from the permuted SIMD register *v* are extracted and appended in the destination register *d_j* (line 12). For example, in Fig. 5(b), we generate *append(d₀, extract(v₀, 0, 2))* for the first fields to do the data transfer marked by solid arrows. Finally, steps (3) to (6) are repeated until all structures are loaded from memory and structure elements are placed in the right SIMD registers and lane position.

If one host SIMD register cannot accommodate more than one structure due to insufficient width (e.g., $\text{NumAS} \leq 1$), the host permutation operations become unprofitable. It is because no multiple elements belonging to the same destination register is within a single host register. In this case, we generate vector loads moving data from memory to host SIMD registers (line 18–26). For each element in the guest register *d_j*,

we calculate which host SIMD register lane it resides in (line 29–30). Then, we append the extracted element to *d_j* (line 31). In Algorithm 1, most steps can be computed at translation time with the information from the decoded guest SIMD structured load/store instructions and the host SIMD organization. Hence, computation of the values, such as *NumAS*, *NumSplit*, *LoadLanes*, *Permute-Mask* etc., do not require generating code at runtime. Only the steps of Load-Vector, Permute-Vector, and Append have code generated in the dynamic binary translator code cache.

3.2.2. Translation using gather/scatter operations

An alternative approach to implementing SIMD structured loads is to use the *vgather* instruction to directly transfer *strided* data into SIMD registers. In this case, explicit data permutation is not required. Algorithm 3 illustrates the translation pseudo code using *vgather*. The

Algorithm 3: Gather/Scatter.

```

input : Stride,
        Destination registers– $D = \{d_0, \dots, d_{i-1}\}$ ,
        Number of lanes to access per guest register–GuestLane,
        Host register width–HostWidth
1 Function Translate-Load(Stride, D, Ptr, GuestLane, HostWidth)
2    $k \leftarrow \text{Permute-Mask}(\text{Stride}, \text{GuestLane})$ 
3    $\text{NumSplit} \leftarrow \lfloor \text{GuestLane} / \text{HostLane} \rfloor$ 
4    $\text{RemLane} \leftarrow \text{GuestLane} \bmod \text{HostLane}$ 
5   foreach  $d_j \in D$  do
6      $\text{Base} \leftarrow \text{GuestLane} \times j$ 
7     if  $\text{NumSplit} > 0$  then
8       for  $i \leftarrow 0$  to  $\text{NumSplit} - 1$  do
9          $k' \leftarrow \text{Extract}(k, \text{Base} + \text{HostLane} \times i, \text{Base} + \text{HostLane} \times (i + 1))$ 
10         $d_j.append(\text{Load-Gather}(\text{Ptr}, k', \text{ElementTy}))$ 
11      end
12    end
13    if  $\text{RemLane} \neq 0$  then
14       $k' \leftarrow \text{Extract}(k, \text{Base} + \text{HostLane} \times \text{NumSplit}, \text{Base} + \text{HostLane} \times (\text{NumSplit} + \text{RemLane}))$ 
15       $d_j.append(\text{Load-Gather}(\text{Ptr}, k', \text{ElementTy}))$ 
16    end
17  end

```

algorithm consists of the following steps: (1) We first calculate the vector of permutation masks (line 2). For example, in Fig. 5(c), the mask *k* is $\langle 0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15 \rangle$. (2) Based on the real number of gather lanes in the host, we calculate the number of *vgathers* to be issued for one guest SIMD register (line 3–4). (3) For each destination register *d_j*, we generate *vgather* instructions to collect structure elements, each associated with the indices from the pre-computed mask *k* (line 5–17).

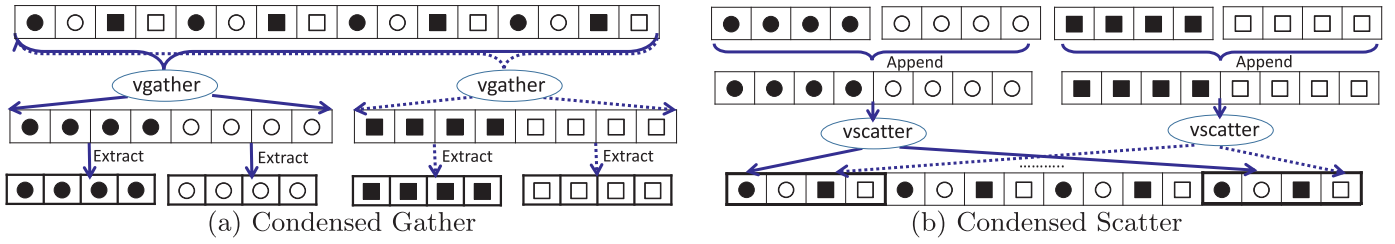


Fig. 7. Translation approaches by condensing multiple guest SIMD registers.

3.2.3. Condensed gather/scatter approach

The translation in Fig. 5(c) generates at least one *vgather*/*vscatter* operation for one guest SIMD register. However, when the number of the guest SIMD lanes is less than that of the host, such gather/scatter approach does not fully utilize the bandwidth of the host gather/scatter instructions. For example, in Fig. 5(c), only half of the host gather bandwidth is used. To solve this issue, we propose an improved algorithm by condensing multiple gather/scatter operations into a single one. As illustrated in Fig. 5(c), four gather operations are required for collecting all elements into corresponding guest registers. In the condensed gather/scatter approach shown in Fig. 7a, we combine the first and second gather operations into one single gather operation. Similarly, the third and the fourth gather operations are combined into another gather operation. As a result, the number of gather operations can be decreased to two. Algorithm 4 shows the pseudo code, and we use Fig. 7a as the

Algorithm 4: Condensed Gather/Scatter.

```

input : Stride, Destination registers  $D = \{d_0, \dots, d_{i-1}\}$ , Number of lanes
        to access per guest register—GuestLane, Host register
        width—HostWidth
1 Function Translate-Load(Stride, D, Ptr, GuestLane, HostWidth)
2    $VF \leftarrow \lfloor \text{HostLane} / \text{GuestLane} \rfloor$ ,  $k \leftarrow \text{Permute-Mask}(\text{Stride}, \text{GuestLane})$ 
3   if  $VF > 1$  then
4      $\text{NumFullLane} \leftarrow \lfloor \text{Stride} / VF \rfloor$ ,  $\text{RemLane} \leftarrow \text{Stride} \bmod VF$ 
5     for  $i \leftarrow 0$  to  $\text{NumFullLane} - 1$  do
6        $k' \leftarrow \text{Extract}(k, VF \times \text{GuestLane} \times i, VF \times \text{GuestLane} \times (i + 1))$ 
7        $v_i \leftarrow \text{Load-Gather}(\text{Ptr}, k', \text{ElementTy})$ 
8     end
9     if  $\text{RemLane} \neq 0$  then
10       $k' \leftarrow \text{Extract}(k, VF \times \text{GuestLane} \times \text{NumFullLane}, \text{GuestLane} \times (VF \times$ 
11         $\text{NumFullLane} + \text{RemLane}))$ 
12       $v_{\text{NumFullLane}} \leftarrow \text{Load-Gather}(\text{Ptr}, k', \text{ElementTy})$ 
13    end
14    foreach  $d_j \in D$  do
15       $\text{idx} = \lfloor j / VF \rfloor$ ,  $\text{offset} = j \bmod VF$ 
16       $d_j \leftarrow \text{Extract}(v_{\text{idx}}, \text{GuestLane} \times \text{offset}, \text{GuestLane} \times (\text{offset} + 1))$ 
17    end
18  else
19     $\text{NumSplit} \leftarrow \lfloor \text{GuestLane} / \text{HostLane} \rfloor$ ,  $\text{RemLane} \leftarrow \text{GuestLane}$ 
20     $\text{mod HostLane}$ 
21    foreach  $d_j \in D$  do
22       $\text{Base} \leftarrow \text{GuestLane} \times j$ 
23      if  $\text{NumSplit} > 0$  then
24        for  $i \leftarrow 0$  to  $\text{NumSplit} - 1$  do
25           $k' \leftarrow$ 
26             $\text{Extract}(k, \text{Base} + \text{HostLane} \times i, \text{Base} + \text{HostLane} \times (i + 1)),$ 
27             $d_j.\text{append}(\text{Load-Gather}(\text{Ptr}, k', \text{ElementTy}))$ 
28        end
29        if  $\text{RemLane} \neq 0$  then
30           $k' \leftarrow \text{Extract}(k, \text{Base} + \text{HostLane} \times \text{NumSplit}, \text{Base} + \text{HostLane} \times$ 
31             $\text{NumSplit} + \text{RemLane})$ 
32           $d_j.\text{append}(\text{Load-Gather}(\text{Ptr}, k', \text{ElementTy}))$ 
33        end
34      end
35    end
36  end

```

example for describing the steps of the Condensed Gather/Scatter approach: (1) We first calculate how many guest SIMD registers one host SIMD register can accommodate and the vector of permutation masks

(line 2). For example, in Fig. 7a the VF (vector factor) is 2, and mask k is $\langle 0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15 \rangle$. (2) If VF is larger than 1, we calculate the number of *vgathers* to be issued (line 4). Otherwise, we back to apply the gather/scatter approach proposed in Section 3.2.2. (3) For VF guest registers, one *vgather* instruction is generated for collecting structure elements (line 5–8). In the example of Fig. 7a, two *vgathers* are generated, and their masks are $\langle 0, 4, 8, 12, 1, 5, 9, 13 \rangle$ and $\langle 2, 6, 10, 14, 3, 7, 11, 15 \rangle$. (4) Because multiple guest registers are accommodated in one host SIMD register, for each destination register d_j , we generate *extracts* to extract the corresponding elements from v_{idx} (line 13–16).

4. Register mapping

The following notations are provided to describe the concept of register mapping in DBT:

- $\text{Length}(G)$: The guest SIMD register width.
- $\text{Length}(H)$: The host SIMD register width.
- $\text{Length}(\text{reg})$: The width of the SIMD register *reg*.
- $\text{SUnit}(\text{reg})$: The set of smallest sub-registers of *reg*.
- $\text{Length}(\text{SUnit}(\text{reg}))$: The width of registers in $\text{SUnit}(\text{reg})$.
- $\text{LUnit}(\text{reg})$: The super-register of the register *reg*.
- $\text{Relative}(\text{reg})$: The set of $\text{LUnit}(\text{reg})$'s sub-registers.

In the following explanatory example, the guest and the host ISA are respectively ARMv7-NEON and x86-AVX2, and the guest instruction `vld3.i8 {d0, d2, d4}, [r0]` is translated. Here, $\text{Length}(G)$ and $\text{Length}(H)$ are 128-bit and 256-bit, respectively. Since $d0$ and $d1$ are sub-registers of register $q0$, $\text{Length}(q0)$ is 128-bit; $\text{LUnit}(d0)$ and $\text{LUnit}(q0)$ are both $q0$; $\text{SUnit}(q0)$ is $\{d0, d1\}$ and $\text{SUnit}(d0)$ is $\{d0\}$; $\text{Length}(\text{SUnit}(q0))$ is 64-bit; $\text{Relative}(d0)$, $\text{Relative}(d1)$ and $\text{Relative}(q0)$ are the same, $\{d0, d1, q0\}$.

The goal of SIMD register mapping is to determine a configuration which maps guest registers to host registers and minimizes data reorganization overhead. The data reorganization operations come from two different sources when translating SIMD structured loads/stores. The first source is from the translation of SIMD structured load/store instructions, where the permutation is conducted to distribute structure elements to their corresponding locations. The second source results from the difference in SIMD register layouts between the guest and the host, specifically due to the name aliasing problem (Section 2.1). One possible approach is to map one guest SIMD super-register to one host SIMD register, e.g., mapping ARMv7 $q0$ to x86-AVX2 *ymm0*. When a following instruction execution operates an aliased sub-register, e.g., manipulating the ARMv7 $d1$ register, its value needs to be extracted from the mapped host register before execution—a source of data reorganization overhead. Another possible mapping is to map one guest SIMD sub-register to one host SIMD register and its sibling sub-register to another host SIMD register. In this case, the two guest SIMD register values need to be extracted and concatenated when their super-register is operated on — this would also introduce data reorganization overhead. Which one of the above mapping schemes is better depends on the usage pattern of the guest SIMD registers (e.g. the define-use relationship of the translation code fragment).

However, a wider SIMD architecture requires complex hardware for efficient data reorganization operations. To simplify SIMD hardware design, microprocessor vendors usually provide efficient data reorganization among a few specific SIMD lanes. Data movement among the other SIMD lanes usually requires multiple hops of data reorganization, thus incurring higher cost. Hence, the register mapper should consider not only the usage of the guest SIMD registers, but also the required data reorganization cost.

In this paper, we describe two approaches to select an efficient register mapping in DBT—an exhaustive search algorithm for performance upper bound estimation which outputs the best solution with minimized data reorganization overhead, and a more practical algorithm which relies on heuristics to achieve good code quality without a prolonged translation.

4.1. Performance upper bound estimation using exhaustive search

To find a best register mapping configuration, we could search all the possible mapping configurations. We collect all guest SIMD registers referenced by the input IRs then divide them into two disjoint sets. One is either defined or used by structured loads/stores, denoted as $\text{UsedReg}(\text{IRs})$. The formal definition of this set is: If $v \in \text{UsedReg}(\text{IRs})$, then $\exists u \in \text{Relative}(v)$ such that u is referenced by at least one structured load/store. The mapping configuration of $\text{UsedReg}(\text{IRs})$ usually needs to be handled delicately. The other set $\text{UnusedReg}(\text{IRs})$ is the complement of $\text{UsedReg}(\text{IRs})$, which contains all guest registers that are not referenced by any structured load/store instruction. We enumerate each mapping approach for these two disjointed sets and define a cost function to evaluate the data reorganization overhead. Finally, the configuration with the minimal cost is selected.

For any guest SIMD register v , if $\text{Length}(v)$ is larger than $\text{Length}(H)$, we map v to host memory because v cannot fit in a single host SIMD register. Otherwise, we can map v to one or more host SIMD registers. Supposing $n = \text{Length}(\text{SUnit}(v))$ and $N = \text{Length}(v)$, all register mapping configurations of v can be denoted as:

$$\text{RegMapSet}(v) = \{ \{v_0^{n-1}, v_n^{2n-1}, \dots, v_{N-n}^{N-1}\}, \{v_0^{2n-1}, v_{2n}^{4n-1}, \dots, v_{N-2n}^{N-1}\}, \dots, \{v_0^{N-1}\} \}$$

where v_a^b represents the bit slice of register v indexing from a to b bit and $\{v_0^{n-1}, v_n^{2n-1}, \dots, v_{N-n}^{N-1}\}$ means mapping bit slices of v to N/n host registers individually. In contrast, $\{v_0^{N-1}\}$ represents mapping v to a single host register. As defined, registers in $\text{UsedReg}(\text{IRs})$ are not involved with structured loads/stores. Therefore, for each register $v \in \text{UsedReg}(\text{IRs})$, we directly map the largest SIMD register in $\text{Relative}(v) \cap \text{UsedReg}(\text{IRs})$ to a dedicated host SIMD register. As a consequence, a unique mapping configuration $\text{DetRegMap}(\text{UsedReg}(\text{IRs}))$ is determined. On the other hand, any register $v \in \text{UsedReg}(\text{IRs})$ may have aliased register u also in $\text{UsedReg}(\text{IRs})$. To prevent mapping aliased register separately, we define:

$$\begin{aligned} \text{LargestRelative}(\text{UsedReg}(\text{IRs})) &= \{u \mid \forall v \in \text{UsedReg}(\text{IRs}), u \\ &\in \text{UsedReg}(\text{IRs}) \cap \text{Relative}(v) \text{ and } u \text{ is the largest} \} \end{aligned}$$

to filter out the non-aliased register set from $\text{UsedReg}(\text{IRs})$, which is used to generate all mapping configurations of $\text{UsedReg}(\text{IRs})$, denoted $\text{Combination}(\text{UsedReg}(\text{IRs}))$:

$$\prod_{v \in \text{LargestRelative}(\text{UsedReg}(\text{IRs}))} \text{RegMapSet}(v)$$

which for $c \in \text{Combination}(\text{UsedReg}(\text{IRs}))$ means one possible configuration of $\text{UsedReg}(\text{IRs})$, and $c \cup \text{DetRegMap}(\text{UsedReg}(\text{IRs}))$ means one possible configuration of input IRs. Thus, the number of possible configurations for the entire IRs is equal to $|\text{Combination}(\text{UsedReg}(\text{IRs}))|$. Consider the example where the guest ISA is ARMv7 and the input IRs contain *StructuredLoad*(4, {d16 d17 d18 d19}, i8, r0). UsedReg is {d16,

d17, d18, d19}. $\text{LargestRelative}(\{d16, d17, d18, d19\}) = \{q8, q9\}$. $\text{Combination}(\text{UsedReg}(\text{IRs}))$ is $\{\{d16, d17, d18, d19\}, \{d16-d17, d18, d19\}, \{d16, d17, d18-d19\}, \{d16-d17, d18-d19\}\} = \{\{d16, d17, d18, d19\}, \{q8, d18, d19\}, \{d16, d17, q9\}, \{q8, q9\}\}$. The number of possible configurations for the input IRs is four.

The final step is to evaluate every possible configuration and output the configuration with minimal data reorganization overhead. However, the data reorganization cost is hard to evaluate. This is because both explicit data reorganization instructions (such as *permute*) and scalar instructions such as *shift*, *and*, or *can* be used to reorganize data. As a result, we use static instruction count of the translated host code as the measurement of data reorganization overhead. Since loops with SIMD instructions typically contain few branches, the dynamic instruction count can be estimated by using the static count.

4.2. The contour algorithm (CT)

Because we leverage the LLVM framework as our dynamic binary translator backend and evaluate register mapping configurations by the host instruction count, we cannot know the cost of mapping configurations until the host binary is generated. Therefore, we need to evaluate all the possible register mapping configurations before selecting the best one, which is very time consuming. Given a sequence of IRs, the algorithm needs to search $(\text{Length}(G)/\text{Length}(\text{SUnit}(v)))^{\#(\text{LargestRelative}(\text{UsedReg}(\text{IRs})))}$. In the worst case, all $(\text{Length}(G)/\text{Length}(\text{SUnit}(v)))^{\#(\text{Guest_SIMD_Registers})}$ configurations should be evaluated. However, after observing the actual behavior of many applications, such as multimedia, signal processing applications and BLAS library routines, we discover that their optimal register mapping configurations exhibit some regularities. The optimal configuration usually maps guest SIMD registers in $\text{UsedReg}(\text{IRs})$ to same-width host SIMD registers. Based on these regularities, we can exclude the combinations in $\text{Combination}(\text{UsedReg}(\text{IRs}))$ which map guest SIMD registers to different-width host SIMD registers and further reduce $|\text{Combination}(\text{UsedReg}(\text{IRs}))|$ to at most $(\text{Length}(G)/\text{Length}(\text{SUnit}(v)))$ combinations. Evaluations in Section 6 confirm that the proposed Contour Algorithm delivers near upper bound performance with very low computation overhead.

5. Reducing data reorganization overhead

Emulating guest structured load/store operations is typically permutation and memory bound so minimizing memory and data reorganization overhead would have a great impact on performance. In this section, we propose the *GuestMemoryFusion* optimization to reduce the data permutations and L1 data cache access that results from the translation of *split memory operation*.

5.1. Split memory operations

Recall that ARMv7 provides name aliases to access any subsets of a SIMD register. Typically, a full-aliased register file has more false dependence issues than partial-aliased registers. For example, when the Q0 register in ARMv7 is written, all the sub-registers, including S0, S1, S2, S3, D0, and D1 are all destroyed. However, for the ARMv8 architecture, only D0 and S0 are destroyed (Fig. 1). Although full-aliased register file would cause more false dependence issues, it offers more flexibility for instruction selection and register allocation. Fig. 8 illustrates this advantage. Fig. 8 performs array addition, $a[i] += b[i]$, and Fig. 8b is the assembly code generated by GCC. Although GCC can vectorize this loop, stores (line 6 and line 7 in Fig. 8b) are still scalar instructions. There are two reasons. One reason is that some ARMv7 micro-architectures handle *vadd.f32 q8, q8, q9* independently between bottom and top half double (D) registers, and the result in bottom of Q8 (D16) is ready earlier than the top of Q8 (D17). The other is that these micro-architectures split one 128-bit SIMD load/store into two 64-bit scalar loads/stores. If there is

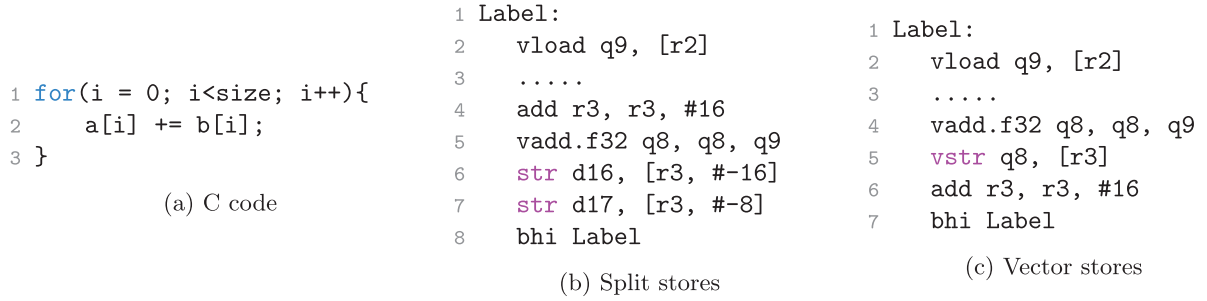


Fig. 8. Array addition.

only one memory read/write port, the throughput of scalar and SIMD loads/stores would be the same. Thus, in Fig. 8, two-scalar-store version can issue the first store instruction once the result of D16 is ready. In contrast, one-vector-store version cannot issue the vector store instruction until the result of Q8 are ready. In our experimental result on Cortex-a53 ARMv7-mode, Fig. 8b does run faster than the vector store version shown in Fig. 8c. Since one SIMD store is split into two scalar stores, it is called split memory operations.

5.1.1. Issues of translating split memory operations

Although the two-scalar-store version outperforms the one-vector-store version under the naive execution on ARMv7 platforms, the split memory operations would cause inefficient translation when the host SIMD registers are not fully aliased, such as x86-AVX2. When translating such split memory operations to x86-AVX2, line 6 and line 7 of Fig. 8b are translated into one extraction instruction taking the higher part of XMM register and two scalar stores. In x86-AVX2, extraction instructions are expensive, and the x86-AVX2 micro-architectures would execute one SIMD memory operation by a single memory transaction. Thus, the two scalar stores translation yields lower performance than using the version of one SIMD store. Moreover, this two scalar store translation may increase the host register pressure, and resulting in host register spill.

5.2. GuestMemoryFusion

To mitigate these performance problems, we develop the GuestMemoryFusion optimization to re-vectorize these split loads/stores. Although LLVM has implemented *Load-store-vectorizer* [21] and *ARMLoadStoreOptimizer* [20] to merge multiple load/store operations, these optimizations cannot solve the problem of split memory operations. *Load-store-vectorizer*, which is designed for GPU kernel, tries to coalesce memory operations. This optimization is usually not beneficial for CPUs, because it possibly generates additional extracting operations which are expensive on most architectures [21]. *Load-store-vectorizer* requires the address of memory operations being contiguous, and the consumers of memory operations should be extracting operations. However, the split loads/stores may have non-extracting consumers, such as inserting and appending operations. Moreover, if the address of split loads/stores is not contiguous, we still can vectorize them by gather/scatter operations. *ARMLoadStoreOptimizer*, designed for the ARM backend is a machine-dependent optimization and works on machine-dependent IRs. It tries to replace multiple scalar loads/stores (e.g., LDR/STR) sharing the same base register with a single multiple-memory operation (i.e., a LDM or a STM). GuestMemoryFusion, by contrast, works on the generalized IRs.

One of the challenges to GuestMemoryFusion is memory aliasing analysis. Considering Fig. 9a, there is one load operation (line 9) between two stores. Even if *r4* and *r3* are respectively resided by different array pointers, the dynamic binary translator cannot fuse line 8 and line 10 due to possible memory aliasing. This possible aliasing comes from the lack of information on *r4* and *r3*. Although the information

that the two pointers in *r4* and *r3* are not overlapped is available at the source code level, such information is not available at the executable (i.e. binary) level. Without such semantic information, the dynamic binary translator cannot figure out that line 8 and line 10 are candidates for memory fusion.

Even with no appropriate semantic information, the dynamic binary translator can still fuse the memory operations with runtime or static dependence checking. For runtime checking, it can detect possible memory dependences, and direct the execution to either scalar code or vector code, as shown in Fig. 9b. Such multi-version code with runtime memory dependence checking is often used in vectorizing compilers to perform conditional vectorization. However, due to the runtime checking overhead, we need to ensure that the performance gain from vector code execution outweighs the checking overhead. Considering the example in Fig. 9b, only one extraction and one store are eliminated with vectorization. If we adopt the runtime checking approach, the checking overhead should be lower than two instructions. However, in our experimental results in Fig. 9, the runtime checking overhead is larger than the profit from eliminating these two instructions. Based on this observation, we attempt to avoid runtime checking. Instead, we will do more analysis statically to enable vector code generation as much as possible.

GuestMemoryFusion is implemented as a LLVM optimization pass. It tries to fuse multiple load/store instructions which write/read data to/from consecutive sub-registers and guarantees no possible memory alias. It operates on basic blocks. For every basic block in the input, GuestMemoryFusion performs the following steps: (1) Scan the whole basic block by starting from the first instruction. If encountering a store or load instruction, Try-Fuse-Store or Try-Fuse-Load is respectively called to perform analysis. (2) The implementation of Try-Fuse-Store and Try-Fuse-Load are similar so we only use Try-Fuse-Store to illustrate the implementation. Starting from this store instruction, Try-Fuse-Store continues scanning the subsequent instructions for collecting stores which read data from consecutive sub-registers (e.g., D16 and D17). Try-Fuse-Store stops the scanning and goes back to step 1 when reaching two situations. One is finding a store/load instruction whose base register is different from the collected store instructions, or finding the modifications to the base register—which indicates a possible alias. The other is finding a store/load which has overlapping offset. If the number of collected stores is larger than 1, Try-Fuse-Store performs re-vectorization. For contiguous offset, these stores are replaced with host contiguous SIMD stores. For non-contiguous offset, these stores are replaced with host scatters. For example, in Fig. 8b, line 6 and line 7 have the same base register *r3*. The two store instructions store 8-byte data to memory, and the difference of their offset is $-8 - (-16) = 8$. Thus, their offset are not overlapping. Because their offset are contiguous, we replace the two stores with a contiguous SIMD store.

6. Evaluation on OpenCV and BLAS

The performance evaluation is conducted in a retargetable DBT system which consists of two-level IR. The QEMU TCG IR (version 2.2) is used as our dynamic binary translator frontend IR, and the LLVM IR is

<pre> 1 Label: 2 vload q9, [r2] 3 4 vload q8, [r3] 5 add r3, r3, #16 6 add r4, r4, #8 7 vadd.f32 q8, q8, q9 8 str d16, [r3, #-16] 9 load d0, [r4, #-8] 10 str d17, [r3, #-8] 11 bhi Label </pre> (a) Scalar code	<pre> 1 Label: 2 vload q9, [r2] 3 4 vload q8, [r3] 5 add r3, r3, #16 6 add r4, r4, #8 7 vadd.f32 q8, q8, q9 8 if((r3-8, r3) ∩ (r4-8, r4) == ∅){ </pre> (b) Runtime memory checking	<pre> 9 vstr q8, [r3] 10 load d0, [r4, #-8] 11 } 12 else{ 13 str d16, [r3, #-16] 14 load d0, [r4, #-8] 15 str d17, [r3, #-8] 16 } 17 bhi Label </pre>
---	---	---

Fig. 9. Memory dependence checking.

Table 2
Information of benchmarks.

Benchmark	Stride	Ratio: ARMv8	Ratio: ARMv7	Benchmark	Stride	Ratio: ARMv8	Ratio: ARMv7	Benchmark	Stride	Ratio: ARMv8	Ratio: ARMv7
BGR2BGR555	3	4%	4%	BGRA2RGBA	4	16%	23%	caxpy	2	18%	20%
BGR2BGRA	3 and 4	15%	18%	GRAY2BGRA	4	10%	14%	cgbmv	2	–	16%
BGR2RGBA	3 and 4	3%	4%	RGB2BGR565	3	4%	4%	cgemv	2	–	20%
BGRA2BGR555	4	3%	3%	RGBA2BGR	3 and 4	15%	22%	complex_mul	2	27%	21%
				RGBA2BGR565	4	3%	4%				

used as the backend IR (version 3.5). In this two-level IR, the guest binary is first fetched and converted into TCG IR, which is then translated to LLVM IR in the dynamic binary translator backend. Our guest to host register mapping is conducted by first mapping the guest SIMD states to LLVM IR virtual registers, which are in turn mapped to the physical registers on the host. Due to a lack of structured load/store semantics in QEMU TCG, we extend TCG IR with a set of SIMD structured load/store pseudo instructions to facilitate our frontend translation.

The experiments are evaluated with the translations from two guest ISAs, ARMv7 and ARMv8, to three host ISAs, x86-SSE4, x86-AVX2, and x86-AVX512. Furthermore, two register mapping algorithms, the exhaustive search for upper bound estimation (Opt) and heuristic-based Contour algorithm (CT) are also implemented and evaluated. The host platform is Intel® Skylake Core™ i7-7820X CPU, 32GB RAM, running Ubuntu 16.04 LTS (Linux kernel 4.15.0). The L1 cache size is 32KB. We select color space conversion kernels from OpenCV (version 3.1) and complex number computing kernels from the BLAS (Netlib version) library as our benchmark suites since they contain frequent structured load/stores. All benchmarks are compiled with ARM-GCC 5.4 and optimization flags, `-O3 -ftree-vectorize -ftree-slp-vectorize`.

For all experiments, we use the translation of SIMD structured loads/stores to scalar-only instructions (shown in Figs. 5(a) and 6(a)) as the baseline performance for comparison. This is the default translation scheme used in the latest QEMU. Actually, our system also translates other SIMD operations, such as arithmetic, swizzle, and contiguous data access operations to host SIMD instructions. In order to focus on the effectiveness of our proposed algorithms, we only report the performance impact of translating structured loads/stores. That is, all SIMD instruction translation between baseline translation and enhanced translation are the same except for structured load/store instructions. Most of the multimedia libraries in OpenCV compute pixels, which consist of 3 or 4 structure elements (RGB or RGBA respectively). In other words, their strides are 3 and 4. Hence, we select the BLAS complex number computing kernels, which are primarily stride-2 cases. Table 2 lists detailed benchmark information, including the strides and the ratio of SIMD structured load/store instructions for ARMv7 and ARMv8. The ratio of SIMD structured loads/stores is mea-

sured as: $Ratio = \#Dynamic\ SIMD\ structured\ load/store\ instructions \div \#Dynamic\ total\ instructions$.

ARMv8 and ARMv7 have similar ratios for most benchmarks except for BGRA2RGBA, RGBA2BGR and complex_mul. This is because benchmarks BGRA2RGBA and RGBA2BGR in ARMv7 use SIMD structured loads/stores of double-spacing mode [3], which results in more SIMD instructions generated than in ARMv8. For complex_mul, more instructions are used for address computation in ARMv7 binary than ARMv8, which results in a higher total instruction count in ARMv7 and thus a smaller ratio.

6.1. Overall performance

Fig. 10 shows the overall benchmark performance. The results of two BLAS benchmarks, cgbmv and cgemv, are not shown for ARMv8 due to a segmentation fault generated during QEMU emulation. By translating the structured loads/stores by SIMD instructions, we can replace several scalar loads/stores with the SIMD loads/stores, thus reducing memory traffic. Fig. 11 shows the normalized L1 data cache access count over the baseline translation. The Opt and CT algorithms have similar outcomes so we only show the result of CT. On average, the speedup of ARMv8 translation is greater than that of ARMv7. The reason is that the SIMD register layout of ARMv7 is rather different from that of the x86 host (Fig. 1), and more data reorganization instructions are required to emulate ARMv7 SIMD instructions. The result also shows that no noticeable improvement is achieved with AVX512 compared to AVX2, and SSE4 outperforms the AVX512/AVX2 hosts in some benchmarks. However, in terms of L1 Dcache access count reduction, translation with AVX512 is generally better than that with AVX2/SSE. This is because the translation of SIMD structured loads/stores to scalar-only instructions is permutation and memory bound. For an efficient translation, we need to optimize not only memory accesses but also the permutation operations. Typically, based on the vector length, AVX512 is expected to outperform AVX2, and AVX2 is expected to outperform SSE. For basic operations such as add, sub, and mul, more lanes would yield greater speedup. However, the performance impact of a set of permutation instructions creates some complications. For example, AVX512 permutation instructions permute the first 128-bit, the second 128-bit,

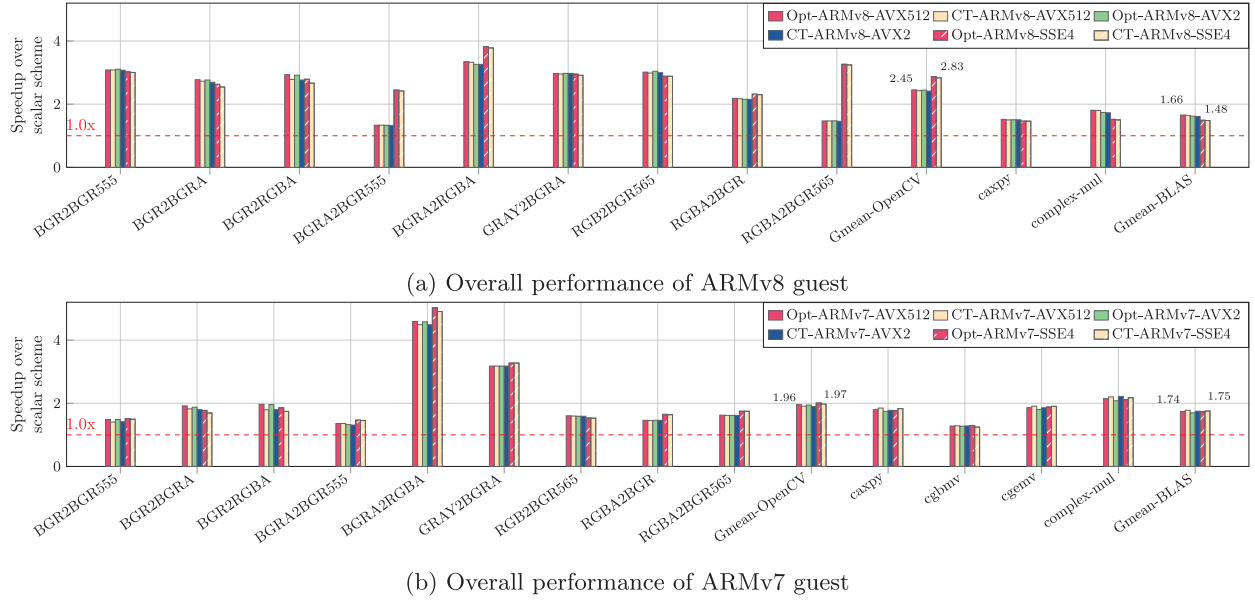


Fig. 10. Overall performance of OpenCV and BLAS benchmarks.

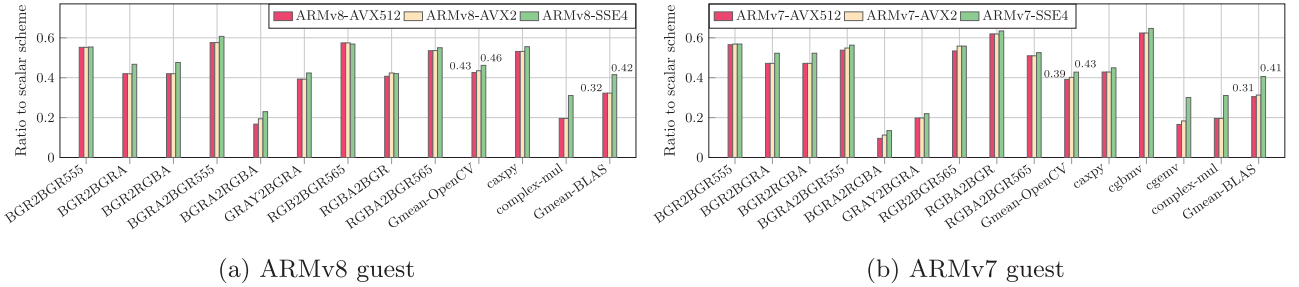


Fig. 11. Ratio of L1 Dcache access count (Load + Store).

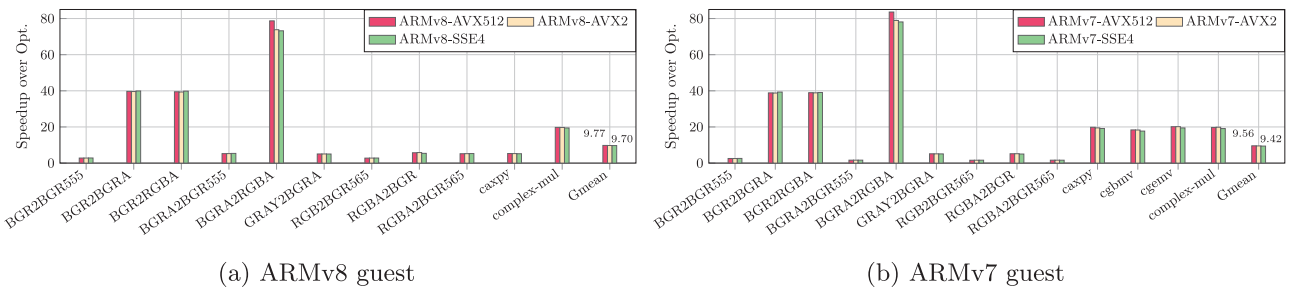


Fig. 12. Compilation time improvement.

the third 128-bit, and the fourth 128-bit parts separately. Therefore, to permute 512-bit data, compilers (programmers) need to apply various tricks and generate more than one instruction [14,30].

Among the benchmarks, kernel BGRA2RGBA has the most outstanding performance. It achieves up to 5.03x speedup and reduces up to 90% of the L1 Dcache accesses. There are two main reasons. First, the ratio of SIMD structured loads/stores is relatively higher in BGRA2RGBA. Second, it operates structured data of stride 4 and 8-bit element size. The baseline translation generates a large sequence of host scalar instructions, which results in large code size and slows down the execution. In contrast, our translation approach generates SIMD code, and thus achieves significant speedup and L1 Dcache accesses reduction.

Fig. 12 shows compilation time improvement when conducting register mapping—comparing CT over Opt. Table 3 shows the absolute compilation time of CT and Opt. For benchmark BGRA2RGBA, its structured

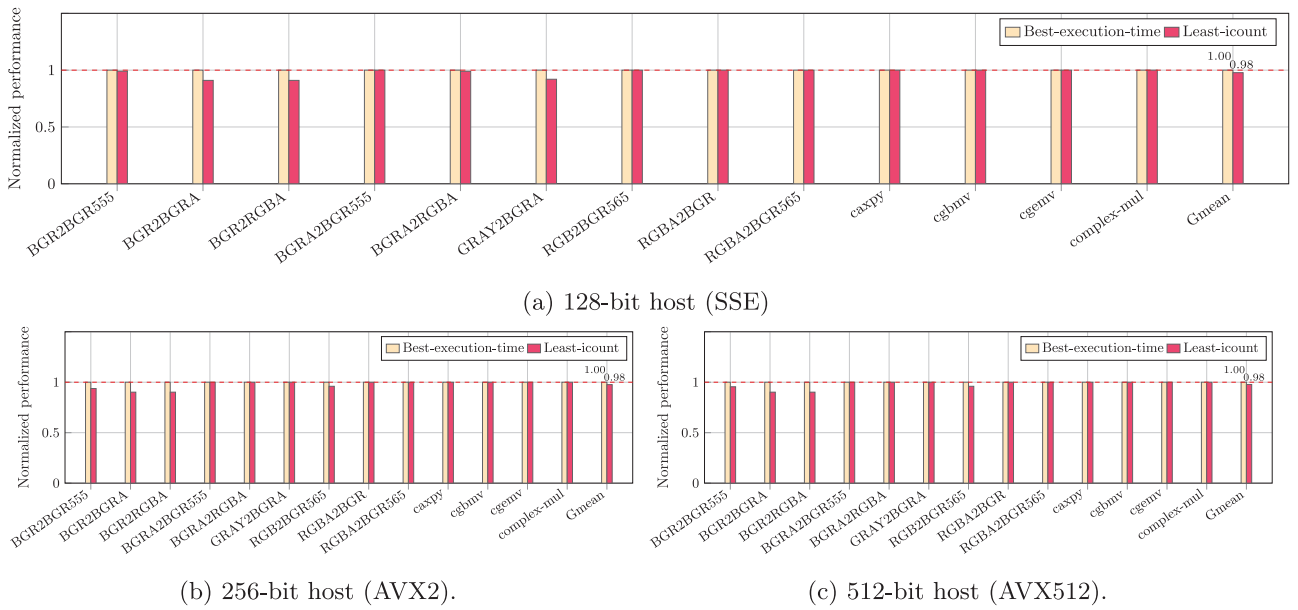
load/store instructions operate 8 SIMD registers, in which the number of register mapping combinations is respectively 256 and 2 for the Opt and CT algorithm. Therefore, CT algorithm can significantly reduce the compilation time. For all benchmarks, the CT algorithm can achieve an average improvement of 9.77x and 9.56x respectively to the ARMv8 and ARMv7 guest.

6.1.1. Cost model validation

In this paper, we use static instruction count of the translated host binary to evaluate the cost of a register mapping configuration. To validate our cost model, we report the performance comparison of the *best-execution-time* register mapping configuration and the configuration selected by our Opt register mapping algorithm, as shown in Fig. 13. In Fig. 13, translation from ARMv7 to x86-SSE, x86-AVX2, and x86-AVX512 are reported. The baseline is the best-execution-time register

Table 3The absolute compilation time (10^{-6} s) of CT and Opt.

Benchmark	ARMv8						ARMv7					
	AVX512		AVX2		SSE		AVX512		AVX2		SSE	
	Opt	CT	Opt	CT	Opt	CT	Opt	CT	Opt	CT	Opt	CT
BGR2BGR555	44,636	16,002	44,610	15,978	47,116	16,648	41,173	16,150	41,136	16,311	42,936	16,641
BGR2BGRA	654,708	16,494	654,309	16,520	722,582	18,105	756,745	19,480	758,135	19,556	825,806	21,006
BGR2RGBA	655,297	16,607	654,523	16,640	723,084	18,163	757,384	19,440	756,668	19,465	822,724	21,059
BGRA2BGR555	138,686	26,485	138,924	26,465	134,666	25,134	35,005	21,608	35,049	21,638	35,403	21,774
BGRA2RGBA	2,078,841	26,413	1,883,468	25,518	1,890,940	25,830	1,808,759	21,635	1,638,771	20,763	1,657,654	21,221
GRAY2BGRA	57,577	11,397	56,861	11,257	58,010	11,534	61,106	11,875	60,736	11,867	61,152	12,017
RGB2BGR565	45,306	16,443	45,402	16,389	47,675	17,014	24,724	15,684	24,773	15,675	26,015	16,203
RGBA2BGR	164,439	28,755	157,023	26,945	141,285	26,187	192,777	37,300	192,817	37,160	182,878	36,425
RGBA2BGR565	97,356	18,793	97,606	18,701	92,088	17,434	25,172	15,594	25,215	15,706	25,275	15,774
caxpy	79,319	14,963	79,055	15,048	75,963	14,665	263,812	13,312	262,917	13,497	248,158	12,958
cgbmv	–	–	–	–	–	–	285,159	15,530	288,065	15,735	266,656	15,056
cgemv	–	–	–	–	–	–	254,149	12,627	256,026	12,700	238,448	12,253
complex_mul	256,150	12,969	256,757	13,025	244,735	12,585	266,489	13,513	269,502	13,577	252,110	13,195

**Fig. 13.** Performance of least-icount configuration.

mapping configuration. Best-execution-time configuration is the configuration that have the shortest execution time among all the possible register mapping configurations, $c \in \text{DetRegMap}(\text{UsedReg}(\text{IRs}))$, where $c \in \text{Combination}(\text{UsedReg}(\text{IRs}))$ defined in Section 4.1. For example, BGRA2RGBA has 256 possible register mapping configurations. The best-execution-time configuration achieves 5.07x speedup over the scalar translation, and the configuration selected by our optimal register mapping algorithm achieves 5.03x speedup. After normalizing to the best-execution-time configuration, the Least-icount configuration bar on BGRA2RGBA of Fig. 13a is 0.99 (5.03/5.07).

The reason why some benchmarks cannot reach one is that evaluating code quality by instruction count is hard to take the impact of multiple instruction issuing and out-of-order execution into consideration. For example, in BGR2BGRA of Fig. 13a, our Opt register mapping algorithm actually selects the configuration with least instruction count but its instruction-per-cycle is smaller than best-execution-time configuration. Therefore, best-execution-time configuration still runs faster. Although least-icount configuration cannot run as fast as best-execution-time configuration for some benchmarks, their values shown in Fig. 13 are very close to 1. Over all benchmarks, the geometry mean is up to 0.97, which has demonstrated that our cost model is competent and near optimal performance.

7. Evaluation on AoS-to-SoA and SoA-to-AoS Kernels

Since ARM NEON only supports a few small strides (2, 3 and 4), it is also interesting to study the translation performance of arbitrary strides. To do this, we directly generate instructions in the LLVM IR to emulate SIMD structured loads/stores of arbitrary strides. Because x86 only supports gather/scatter instructions whose element size is either 32-bit or 64-bit, the OpenCV benchmark (mostly uses 8-bit structure elements for pixel colors) is not suitable for the evaluation of the gather/scatter schemes. Instead, we design test cases using array of structure (AoS) to structure of array (SoA) conversion for *vldn* and SoA to AoS for *vstn*. The data elements of structures are 32-bit elements. We set the stride from 2 to 16 (x-axis) and fix the number of structured loads/stores to 128. Therefore, total memory sizes from 1 (stride is 2) to 8 (stride is 16) KB data are transferred. In Section 2.2, we discuss that the recent compilers are able to generate structured load instructions for patterns with partially-used elements. Thus, this section also investigates the translation performance of the partially-used patterns.

7.1. Fully-used structured loads

Fig. 14 shows the achieved speedup of different structured load translation approaches. The baseline for comparison is the scalar

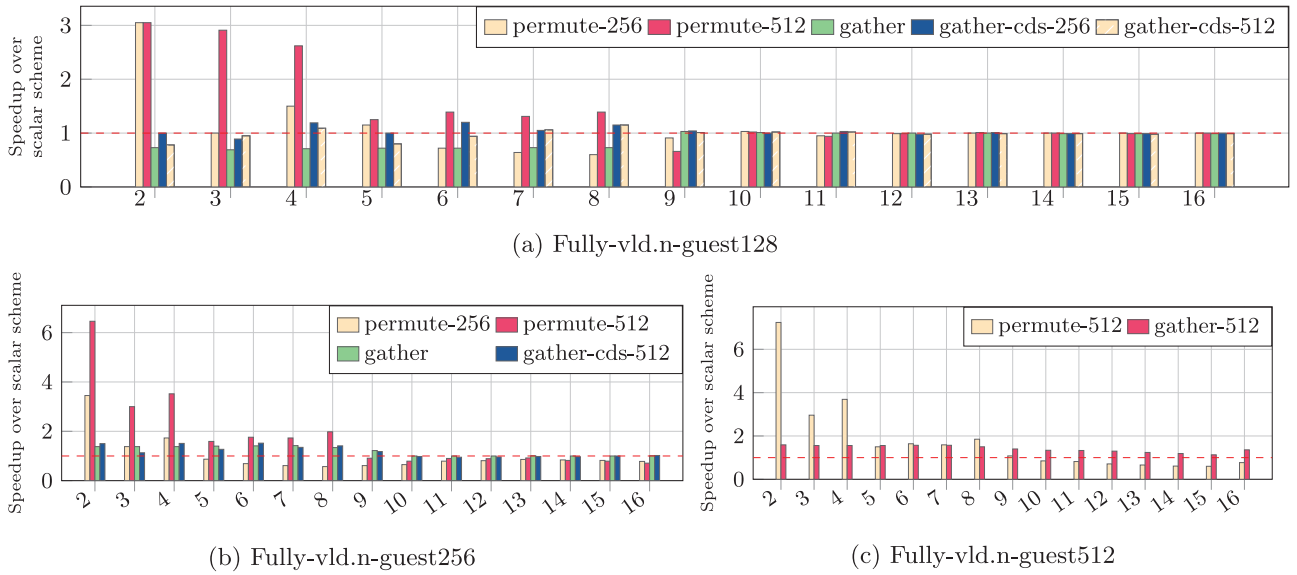


Fig. 14. Performance of fully-used *vld.n* (*X* bar is the *stride*.).

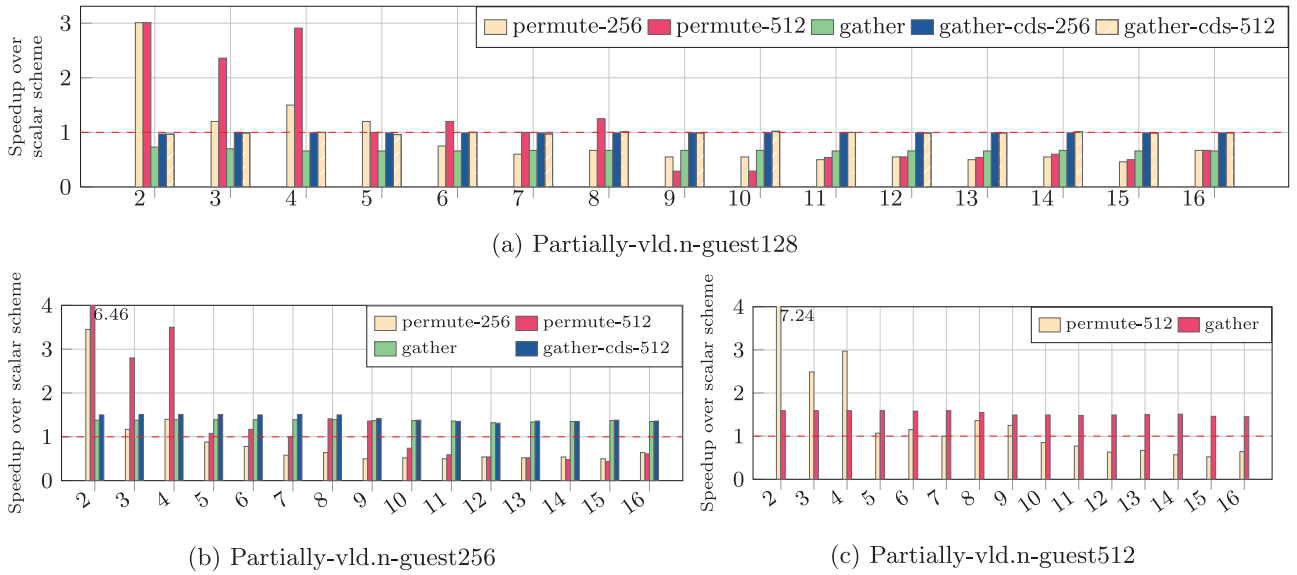
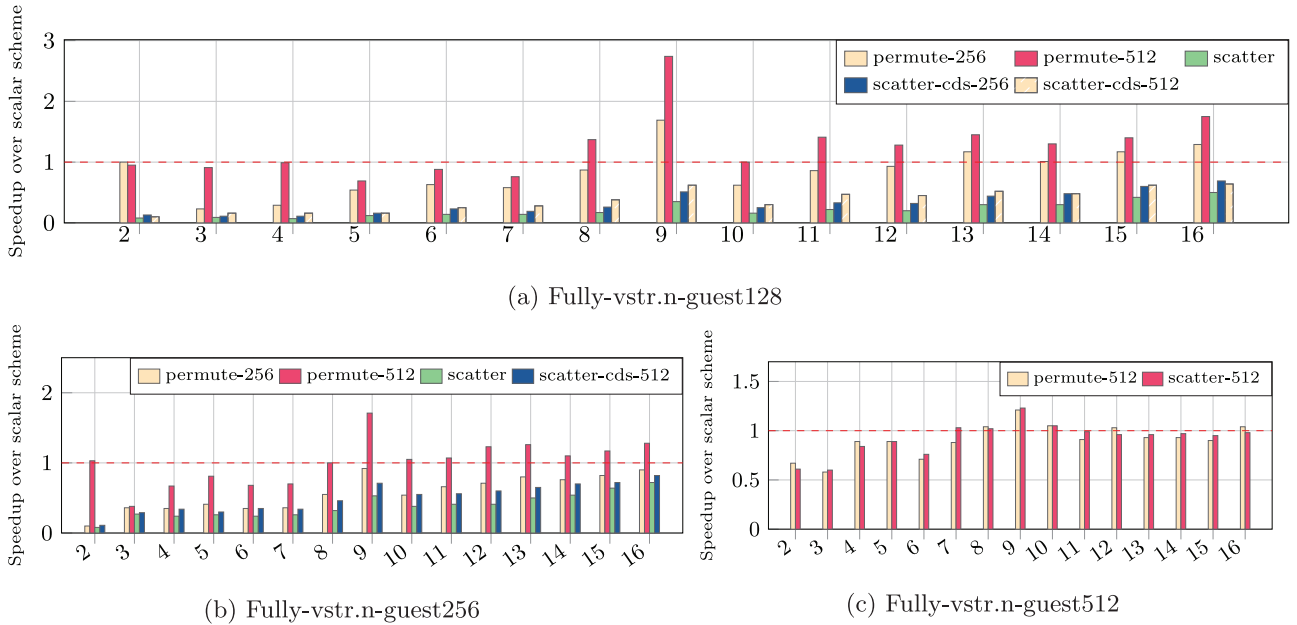
translation, using scalar instructions to move data from memory to SIMD registers, as shown in Fig. 5a. In Fig. 14a, we set the guest register length to 128-bit which is same as the ARM NEON register. The *permute-256* means using *contiguous load and permute (permute)* approach, and host register length is 256-bit. Similarly, *gather* means using gather approach, and *gather-cds* means using condensed gather approach. For permute approach of Fig. 14a, its speedup is larger than 3x at stride 2, and decreases as the stride (size of the structure) increases. This is because permute approach requires multiple structures to be accommodated in one host SIMD register so as to permute elements belonging to the same guest SIMD register across different structures, as discussed in Algorithm 1. In other words, the more structures one host SIMD register can accommodate, the less permutations are needed. When host register length is 512-bit, permute approach can simultaneously permute 8 structures ($512\text{-bit}/(32\text{-bit} \times 2)$) for stride 2 and 2 structures for stride 8. When the stride is larger than 8, permute approach goes to Algorithm 1 line 17. Thus, the performance degrades as the stride (size of structure) increases. For gather approach, it runs slower than permute approach for all strides except stride 9 and 11. According to the Intel optimization manual [13], gathering more elements with a gather instruction helps amortize the latency and increases the gather throughput. For instance, the latency of two 128-bit gathers is larger than one equivalent 256-bit gather. Based on Algorithm 3, every gather instruction gathers 128-bit data, the length of guest SIMD registers, so the gather bandwidth is not fully utilized. To increase gathered elements of a gather instruction, we propose the gather-cds approach. The gather-cds generally runs faster than gather approach, but still running slower than permute approach if stride is less than 8. According to [13], access patterns with small strides favors non-gather instructions. In the experiment results, gather-cds approach is preferred when the stride is too large for the host SIMD register to accommodate multiple structures.

Recently, hardware vendors continuously increase the SIMD register length so it is interesting to study the translation performance when guest SIMD register length goes longer. Fig. 14b shows the translation performance of 256-bit guest SIMD register length. Because one structured load instruction can load *Guest Register Length/Element Size* structures, a structured load with 256-bit SIMD register length loads twice structures of the one with 128-bit SIMD register length. Thus, the parallelisms exploited by 256-bit-register structured loads are more than 128-bit-register structured loads. Consequently, the performance shown in Fig. 14b is better than Fig. 14a. Fig. 14c shows the translation per-

formance of 512-bit guest SIMD register length. In this case, gather-cds approach is not required because the guest and host have the same SIMD register length, and the gather bandwidth can be fully utilized. Therefore, the gather approach runs faster than scalar translation among all strides. For the permute-256/permute-512 approach in Fig. 14b and c and permute-256 in Fig. 14a, we observe that the performance of stride-4 has larger speedup than stride-3. This is because the stride of stride-3 kernel is not power of two, thereby requiring more permutation operations than stride-4. Both the stride-3 and stride-4 kernels are permutation bound (using port 5 of the Skylake processor), so the additional permutations further decrease the speedup. However, for permute-512 in Fig. 14a, the guest SIMD register width is 128-bit and the host SIMD register width is 512-bit. According to line 4 of Algorithm 1, both stride-3 and stride-4 require only one permutation operation, and they are memory bound (port 4). Because the stride-4 kernel has more memory operations than the stride-3 kernel, stride-4 has lower speedup than stride-3. Based on the observation, the gather or gather-cds approach is preferred when the stride is too large for the host SIMD register to accommodate multiple structures.

7.2. Partially-used structured loads

Fig. 15 shows the performance that only the first and second loaded structure elements are used. Different to the fully-used setting, the accessed data in partially-used cases are sparse. Therefore, the permute approach, which continuously loads data from memory to SIMD registers, performs poorly due to redundant loads. Hence, the permute approach in Fig. 15 has lower speedup compared with that in Fig. 14. For gather approach, we require two gather instructions regardless of the stride, so the performance does not change when varying the strides. We can observe that the gather approach in Fig. 15 has better speedup than Fig. 14. The memory accesses in Fig. 15 are sparser than Fig. 14, and thus the advantage of gathering sparse memory of gather instructions can be exploited. For gather-cds approach, it condenses the two gather instructions generated by gather approach into one single gather instruction. Therefore, in Fig. 15, the condensed gather approach generally outperforms the gather approach. Based on the observations, most the translation approaches get better performance when the length of guest SIMD register increases. Moreover, when the number of used structure elements decreases (i.e., from *fully-used vld.n* to *partially-used vld.n*),

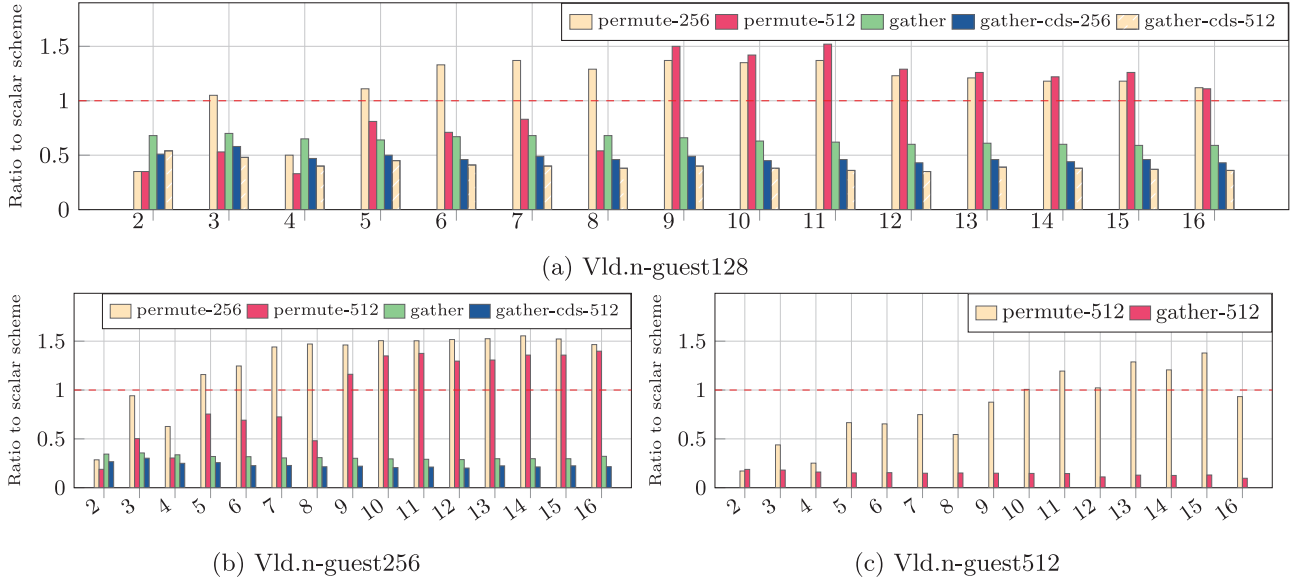
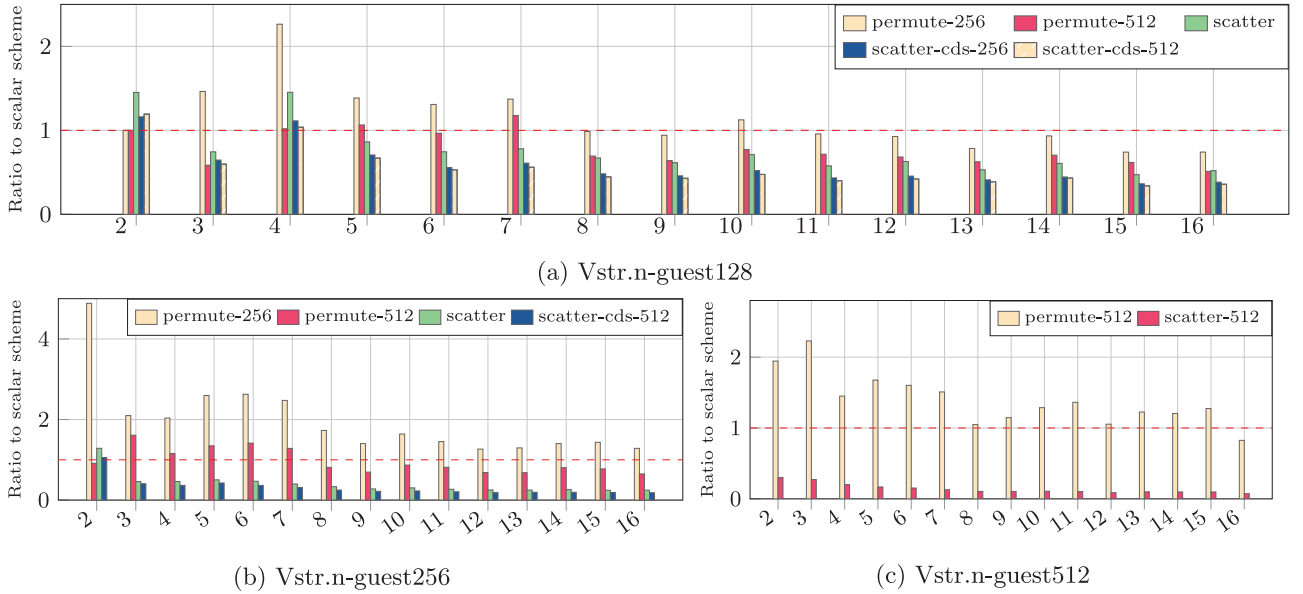
Fig. 15. Performance of Partially-used *vld.n* (*X* bar is the *stride*.).Fig. 16. Performance of *vstr.n* (*X* bar is the *stride*.).

the gather and gather-cds approaches get better performance, where the permute approach has the opposite trend.

7.3. Structured stores

Fig. 16 shows the achieved speedup of different structured store translation approaches. The baseline for comparison is the scalar translation, using scalar instructions to move data from SIMD registers to memory, as shown in Fig. 6a. In Fig. 16a, *Fully-vstr.n-guest128*, means we set the guest register length to 128-bit. For the permute approach, we can observe that the performance trend is inconsistent with the structured load translation shown in Fig. 14. Structured stores whose strides are larger than 8 have better speedup than those with strides smaller than 8. Different with structured load, structured store is to interleave the arranged data in multiple guest SIMD registers, as shown in Fig. 6b. Since the guest SIMD register size is smaller than the host's, we can al-

ways compose multiple guest SIMD register data into the longer host SIMD register, regardless of the stride value of the structured store instruction. Therefore, we can benefit from the intra-SIMD permutation to interleave the data within the host SIMD register. As a result, the speedup increases with the stride value, as shown in Fig. 16a. On the other hand, if the guest SIMD register becomes longer, the number of guest registers accommodated in one host SIMD register becomes fewer. Therefore, more permutations are needed, and the speedup decreases as the guest SIMD register length increases. For scatter-cds and scatter approaches, their speedup increases as the guest SIMD register length increases. The reason is that the longer SIMD register length increases the utilization of scatter bandwidth. Although scatter-cds has increased the scattered elements of a scatter instruction by condensing multiple scatter instructions into a single one, longer guest SIMD registers still help scatter-cds reduce the *Append* operations, as shown in Fig. 7b. Compared with the permute approach, the speedup of scatter-cds and scatter

Fig. 17. Ratio of generated code size for *vld.n* (*X* bar is the *stride*.).Fig. 18. Ratio of generated code size for *vstr.n* (*X* bar is the *stride*.).

is not significant. It is because both approaches suffer from frequently Microcode Sequencer (MS) Switch [13]. The MS is used for complex instructions as well as micro-code assists, and scatter instructions would trigger the MS.

7.4. Generated code size

Figs. 17 and 18 show the generated code size of different translation schemes. We report the results of three different guest SIMD register length, including 128-bit, 256-bit, and 512-bit; we use the scalar translation as the baseline for comparison. For instance, in Fig. 17b, gather-cds-512 bar with stride 2 is 0.27. It means $(\text{code size of gather-cds-512})/(\text{code size of scalar schemes}) = 0.27$ when the stride is 2. In overall, gather-cds and scatter-cds approaches generate the least code size except the small strides, such as 2. Actually, in cases of small strides, all of the approaches generate small code size due to the small size of structures. Therefore, gather-cds and scatter-cds approaches are good options when optimizing for code size.

8. Evaluation of GuestMemoryFusion optimization

8.1. Overall performance

Variance between guest and host SIMD register structures may result in additional data reorganizing and memory operations. In Section 5, we propose the GuestMemoryFusion algorithm to handle the split guest loads/stores problem. This algorithm fuses split loads/stores into vector loads/stores, hence reducing the L1-data-cache access and data reorganization overhead. To quantify the performance of GuestMemoryFusion, we report its impact on overall speedup, number of micro-operations dispatched to scheduler port 5, and L1-data-cache access in Fig. 19. Intel Skylake micro-architecture has 8 scheduler ports. Port 5 handles integer ALU, fast LEA, vector ALU, gathers, and vector permutations such as extraction and insertion [13], so we use the number of micro-operations dispatched to port 5 as one of the evaluation metrics. For the benchmarks, we select kernels from OpenCV,

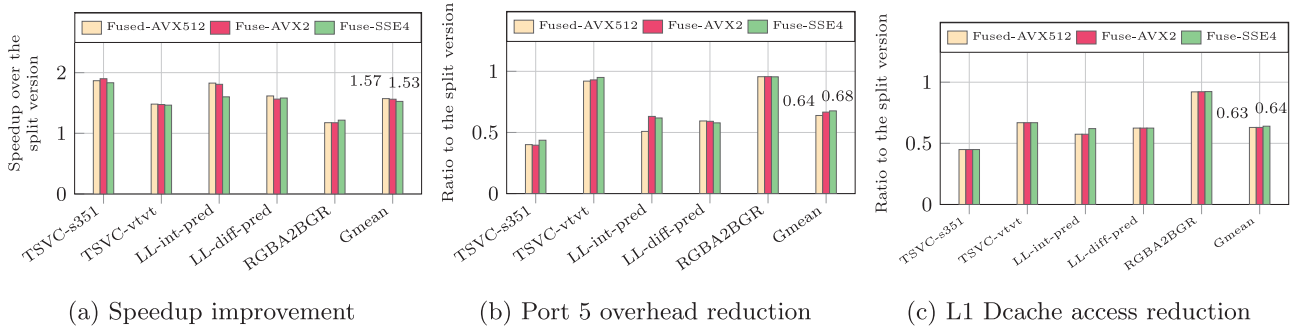


Fig. 19. Performance of GuestMemoryFusion optimization.

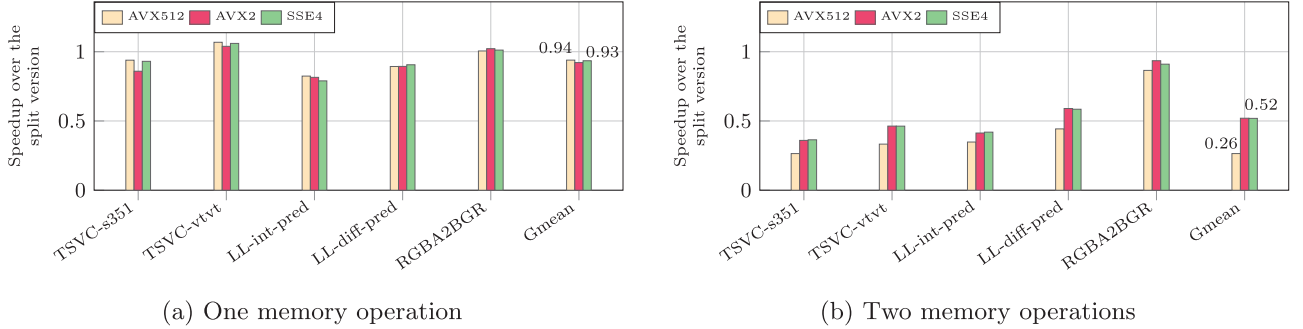


Fig. 20. Runtime checking overhead over execution time.

Test Suite for Vectorizing Compilers (TSVC) [22], and Livermore Loops (LL) [23]. In Fig. 19a, TSVC-s353 and LL-int-pred have the highest speedup. It is because the two benchmarks have relatively higher split loads/stores ratio than other benchmarks. For benchmark LL-int-pred, x86-SSE4 host has a performance gap to x86-AVX512 and x86-AVX2 hosts. This is because unlike x86-AVX512 and x86-AVX2, x86-SSE4 does not support the fused multiply-and-add instructions. Therefore, x86-SSE4 decomposes a guest multiply-and-add instruction into two instructions: a multiplication and an addition operation. In some cases, the decomposition may not result in higher execution time due to the out-of-order execution. However, LL-int-pred has a sequence of multiply-and-add use-definition chain so the out-of-order execution cannot amortize the latency. In Fig. 19c, the L1-data-cache access reduction of TSVC-s351 is lower than the theoretical number, 0.5. The reason is that the host needs temporal registers to hold the values from/to split guest load/store instructions, and thus increases register pressure on the host. In TSVC-s351, GuestMemoryFusion not only re-vectorizes the split load/stores but also reduces the stack reference so the L1-data-cache access reduction is lower than 0.5.

8.2. Runtime checking overhead

Fusing split guest loads/stores is effective to minimize data reorganization, but it could also impose extra overhead if runtime checking is required to guard the fused memory operations against potentially aliasing ones. From the selected kernels with split access, we find that all the split memory operations can be fused at translation time, incurring no runtime overhead. However, we are still curious about the impact from runtime checks. Thus, we manually insert some dummy, non-aliasing memory operation(s) between the split access to force the insertion of runtime checking codes and study its overhead. Figs. 20 to 22 respectively show the measurement results in terms of the execution time, dynamic instruction count, and static instruction count. The baseline is the performance without the GuestMemoryFusion optimization.

We evaluate two scenarios: (1) one memory operation is inserted between the split memory operations, denoted as *one memory operation*,

and (2) two memory operations are inserted, denoted as *two memory operation*. Runtime checking overhead depends on the number of checking pairs. Hence, *two memory operation* would have more overhead than *one memory operation*. As the execution time result shows in Fig. 20a, several benchmarks exhibit performance degradation even if only one memory operation needs runtime checking; such situation becomes even worse when two memory operations are checked, indicating that the imposed overhead is greater than the profits from fusion. Figs. 21 and 22 also show significant instruction count increases over the baseline. The result in this study suggests avoiding runtime checking, and thus, we only fuse split memory operations whose aliasing issue can be resolved statically.

9. Translation to structured loads/stores

The previous sections propose the algorithms for translating structured loads/stores to contiguous SIMD loads/stores with permutations, to gathers/scatters, or to condensed gathers/scatters. This section discusses the reverse direction (i.e., from contiguous SIMD loads/stores with permutations to structured loads/stores, from gathers/scatters to structured loads/stores, and from condensed gathers/scatters to structured loads/stores). For ISAs supporting only contiguous SIMD loads/stores and permutations, the structured load/store patterns are lowered into a sequence of loads/stores and various data-reorganization instructions. The data reorganization instructions can be *shift*, *or*, *and*, *shuffle*, *pack*, *insert*, and so on. Because not only explicit permutation instructions (e.g., *shuffle*, *pack*) but also logic operations (e.g., *shift*, *and*, *or*) can reorganize data, it is difficult to derive rules for automatically recovering the structured loads/stores. An alternative approach is annotating the semantics of structured loads/stores in the binary. Then, dynamic binary translators can reverse the structured loads/stores by following the annotations. On the other hand, reversing structured loads/stores from the gathers/scatters or condensed gathers/scatters seems easier because we can derive the access patterns by analyzing the mask and index registers of the gather/scatter instructions. However, the value of the index registers may be loaded from memory, which

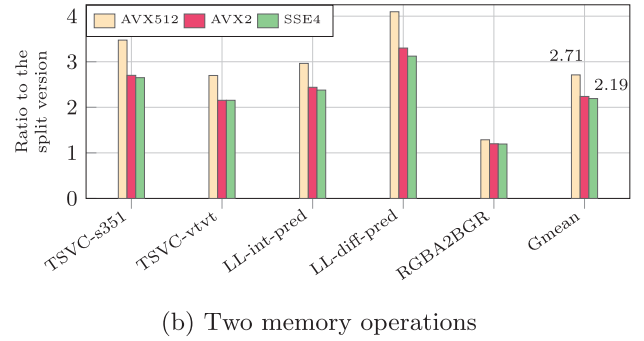
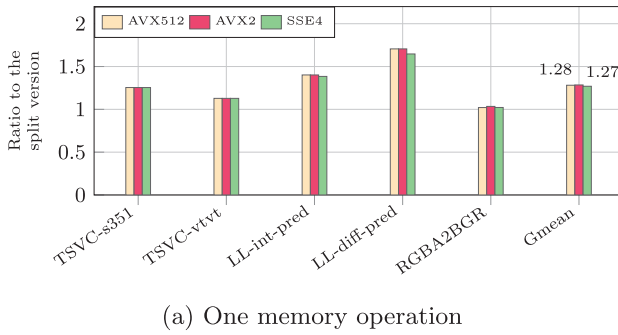


Fig. 21. Runtime checking overhead over dynamic instruction count.

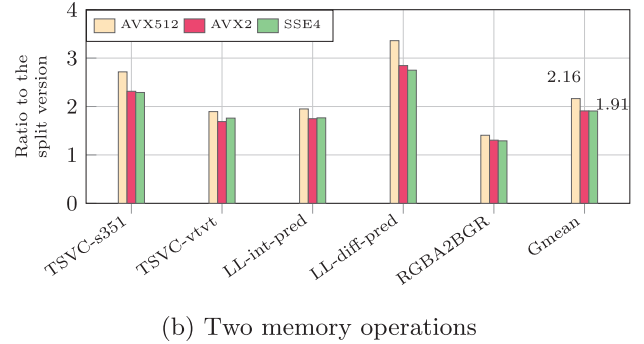
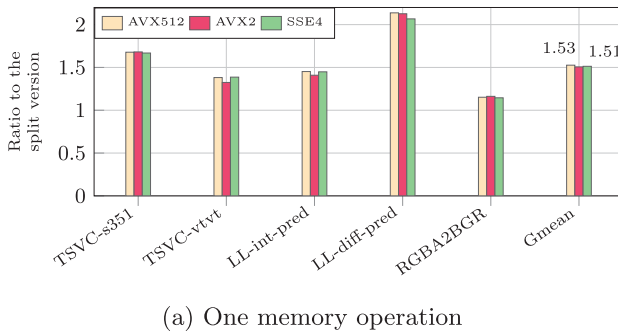


Fig. 22. Runtime checking overhead over static instruction count.

makes it difficult to determine whether the access patterns are structured SIMD operations. Instead, we can apply the annotation approach, which annotates the index values to help dynamic binary translators analyze the structured operations.

10. Related work

SIMD architectures offer greater power and performance efficiency by exploiting data-level parallelism. Earlier SIMD ISAs lack non-contiguous memory access instructions. SIMD-aware compilers often vectorize only loops with contiguous memory access patterns. For compilers exploiting SLP (Superword Level Parallelism), the primary targets are also code with contiguous memory accesses [16,29]. Some recent compilers studied vectorization opportunities with non-contiguous data access patterns [26]. They showed that certain interleaved access patterns, e.g., strides with power of 2, could be successfully vectorized with contiguous memory operations plus data reorganization operations such as shuffle and permutation. Following the advancements, subsequent researchers have improved vectorization techniques by supporting more non-contiguous memory access and irregular patterns [1,2,17,18,28,31,33,34]. Anderson et al. [2] propose algorithms for vectorizing strides with power-of-2 and non-power-of-2. Similar to our work, their algorithms work on general IRs but have three major differences against ours. First, their algorithms do not include the cases that size of *Accesses Group* is larger than hardware SIMD register length. *Accesses Group* is defined by Anderson et al. [2], which means accesses to the same array with the same direction (read or write) and a shared stride. Second, they focus on vectorizing the source code by contiguous memory operations with permutations, and thus do not present algorithms for hardware gathers/scatters instructions. Third, their algorithms target for source code so they do not have to handle the issues resulted from the variances between guest and host ISA. Aleen et al. [1] is another work discussing optimizations for gather/scatter and overlapping contiguous load. However, they build the algorithms on the *Oracle Database* [1] rather in general IRs.

SIMD hardware evolves in terms of register width, register addressing ability, memory addressing, as well as functionality, and thus making application binary migration from one SIMD ISA to another difficult. To mitigate this problem, Nathan Clark et al. propose Liquid SIMD [5]. The Liquid SIMD compiler compiles source code into scalar instructions and maps them to SIMD instructions by special hardware units at runtime. Different from our DBT work that directly handles executables, Liquid SIMD processes high-level language source code.

DBT technique has been widely used in application migration, program analysis, computer security enforcement, and runtime optimization. In particular, Cross-ISA DBT is a very common approach for virtualization such as the widely used Android emulator, Genymotion and BlueStacks, which run ARM applications on x86 machines. For DBT works addressing translation overhead optimization, CoDBT [9] and SINOF [10] are related works. CoDBT is a SW/HW collaborative translation system, which uses a specific hardware accelerator to handle the time-consuming operations. SINOF is a dynamic-static combined DBT framework, which profiles the binary in the runtime phase and performs optimizations at the static phase. For DBT works addressing SIMD translation, Li et al. [19] proposed a tracking algorithm to reduce the SIMD type conversions. Hallou et al. [11] proposed a same-ISA dynamic binary translator which converting short-SIMD binary to long-SIMD code. However, few DBT works have addressed the translation of non-contiguous data access instructions and issues resulted from different register structures.

11. Conclusion

Structured loads/store is a type of non-contiguous memory operations targeting multimedia, signal processing, mathematical, and 2D matrix transposition applications. Current DBT systems have yet to explore efficient translation of structured loads/stores. Because emulating guest structured load/store operations is typically permutation and memory bound, how to avoid extra memory and data reorganization operations has critical impact on translation performance. We present a

comprehensive translation approach, including *Contiguous Loads/Stores and Permutation, Gather/Scatter, and Condensed Gather/Scatter*. Moreover, we design a heuristic-based SIMD register mapping algorithm and related optimizations to reduce data permutation and memory operations in generated code. To the best of our knowledge, this is the first work to address such issues for translating non-contiguous memory operations. Actually, the proposed register mapping mechanism and optimizations are not limited to structured loads/stores translation; they can also be extended to handle translation tasks where the SIMD register structures between guest and host are different. We have implemented our designs in a QEMU+LLVM two-level-IR DBT system. Using selected OpenCV and BLAS routines as benchmarks, our system can achieve a maximum speedup of 5.27x, and an average speed improvement of 3.17x, when migrating ARM NEON executables to x86 Skylake machines. We have also investigated how SIMD register length, strides, and different structured load use cases impact on the translation performance and code size. These results provide guidelines on whether to generate scalar loads/stores, contiguous loads/stores plus permutations, gathers/scatters, or gathers/scatters plus extractions/appends (condensed gather/scatter approach) code under different situations to achieve better performance on the host machine.

Conflicts of Interest

None.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:[10.1016/j.sysarc.2019.07.008](https://doi.org/10.1016/j.sysarc.2019.07.008).

References

- [1] F. Aleen, V.P. Zakharin, R. Krishnaiyer, G. Gupta, D. Kreitzer, C.L. Jr., Automated compiler optimization of multiple vector loads/stores, *Int. J. Parallel Program.* 46 (2) (2018) 471–503, doi:[10.1007/s10766-016-0485-7](https://doi.org/10.1007/s10766-016-0485-7).
- [2] A. Anderson, A. Malik, D. Gregg, Automatic vectorization of interleaved data revisited, *ACM Trans. Archit. Code Optim. (TACO)* 12 (4) (2016) 50.
- [3] ARM, Arm Architecture Reference Manual, ARM Corporation, 2016.
- [4] F. Bellard, QEMU, a fast and portable dynamic translator, in: *Proceedings of the FREENIX Track: 2005 USENIX Annual Technical Conference*, April 10–15, 2005, Anaheim, CA, USA, 2005, pp. 41–46.
- [5] N. Clark, A. Hormati, S. Yehia, S.A. Mahlke, K. Flautner, Liquid SIMD: abstracting SIMD hardware using lightweight dynamic mapping, in: *13th International Conference on High-Performance Computer Architecture (HPCA-13 2007)*, 10–14 February 2007, Phoenix, Arizona, USA, 2007, pp. 216–227, doi:[10.1109/HPCA.2007.346199](https://doi.org/10.1109/HPCA.2007.346199).
- [6] F. Franchetti, M. Püschel, Generating SIMD vectorized permutations, in: *Compiler Construction, 17th International Conference, CC 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29, April 6, 2008. Proceedings*, 2008, pp. 116–131, doi:[10.1007/978-3-540-78791-4_8](https://doi.org/10.1007/978-3-540-78791-4_8).
- [7] S. Fu, D. Hong, Y. Liu, J. Wu, W. Hsu, Dynamic translation of structured loads/stores and register mapping for architectures with SIMD extensions, in: *Proceedings of the 18th ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems, LCTES 2017, Barcelona, Spain, June 21–22, 2017*, 2017, pp. 31–40, doi:[10.1145/3078633.3081029](https://doi.org/10.1145/3078633.3081029).
- [8] S. Fu, J. Wu, W. Hsu, Improving SIMD code generation in QEMU, in: *Proceedings of the 2015 Design, Automation & Test in Europe Conference & Exhibition, DATE 2015, Grenoble, France, March 9–13, 2015*, 2015, pp. 1233–1236.
- [9] H. Guan, B. Liu, Z. Qi, Y. Yang, H. Yang, A. Liang, CoDBT: a multi-source dynamic binary translator using hardware/software collaborative techniques, *J. Syst. Archit.* 56 (10) (2010) 500–508, doi:[10.1016/j.sysarc.2010.07.008](https://doi.org/10.1016/j.sysarc.2010.07.008).
- [10] H. Guan, E. Zhu, H. Hu, R. Ma, Y. Yang, B. Wang, SINOF: a dynamic-static combined framework for dynamic binary translation, *J. Syst. Archit.* 58 (8) (2012) 305–317, doi:[10.1016/j.sysarc.2012.05.002](https://doi.org/10.1016/j.sysarc.2012.05.002).
- [11] N. Hallou, E. Rohou, P. Clauss, A. Ketterlin, Dynamic re-vectorization of binary code, in: *2015 International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation, SAMOS 2015, Samos, Greece, July 19–23, 2015*, 2015, pp. 228–237, doi:[10.1109/SAMOS.2015.7363680](https://doi.org/10.1109/SAMOS.2015.7363680).
- [12] C.J. Hughes, Single-instruction multiple-data execution, *Synth. Lect. Comput. Archit.* 10 (1) (2015) 1–121.
- [13] Intel, Intel 64 and IA-32 Architectures Optimization Reference Manual, Intel Corporation, 2018.
- [14] Intel, Intel 64 and IA-32 Architectures Software Developer's Manual, Intel Corporation, 2019.
- [15] S. Kim, H. Han, Efficient SIMD code generation for irregular kernels, *SIGPLAN Not.* 47 (8) (2012) 55–64, doi:[10.1145/2370036.2145824](https://doi.org/10.1145/2370036.2145824).
- [16] S. Larsen, S.P. Amarasinghe, Exploiting superword level parallelism with multimedia instruction sets, in: *Proceedings of the 2000 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Vancouver, British Columbia, Canada, June 18–21, 2000, 2000, pp. 145–156, doi:[10.1145/349299.349320](https://doi.org/10.1145/349299.349320).
- [17] M. Lattuada, F. Ferrandi, Exploiting vectorization in high level synthesis of nested irregular loops, *J. Syst. Archit.* 75 (2017) 1–14, doi:[10.1016/j.sysarc.2017.03.001](https://doi.org/10.1016/j.sysarc.2017.03.001).
- [18] F. Lemaitre, B. Couturier, L. Lacassagne, Cholesky factorization on SIMD multi-core architectures, *J. Syst. Archit.* 79 (2017) 1–15, doi:[10.1016/j.sysarc.2017.06.005](https://doi.org/10.1016/j.sysarc.2017.06.005).
- [19] J. Li, Q. Zhang, S. Xu, B. Huang, Optimizing dynamic binary translation for SIMD instructions, in: *Proceedings of the International Symposium on Code Generation and Optimization*, IEEE Computer Society, 2006, pp. 269–280.
- [20] LLVM, Armloadstoreoptimizer.cpp, LLVM 7.0 (2018a).
- [21] LLVM, Loadstorevectorizer.cpp, LLVM 7.0 (2018b).
- [22] S. Maleki, Y. Gao, M.J. Garzarán, T. Wong, D.A. Padua, An evaluation of vectorizing compilers, in: *2011 International Conference on Parallel Architectures and Compilation Techniques, PACT 2011, Galveston, TX, USA, October 10–14, 2011*, 2011, pp. 372–382, doi:[10.1109/PACT.2011.68](https://doi.org/10.1109/PACT.2011.68).
- [23] F.H. McMahon, The Livermore Fortran Kernels: A computer test of the numerical performance range, Technical Report, Lawrence Livermore National Lab., CA (USA), 1986.
- [24] L. Michel, N. Fournel, F. Pétrot, Speeding-up SIMD instructions dynamic binary translation in embedded processor simulation, in: *Design, Automation and Test in Europe, DATE 2011, Grenoble, France, March 14–18, 2011*, 2011, pp. 277–280, doi:[10.1109/DATE.2011.5763274](https://doi.org/10.1109/DATE.2011.5763274).
- [25] D. Naishlos, M. Biberstein, A. Zaks, Compiler Vectorization Techniques for Disjoint SIMD Architectures, Technical Report, 2002.
- [26] D. Nuzman, I. Rosen, A. Zaks, Auto-vectorization of interleaved data for SIMD, *SIGPLAN Not.* 41 (6) (2006) 132–143, doi:[10.1145/1133255.1133997](https://doi.org/10.1145/1133255.1133997).
- [27] G. Ren, P. Wu, D. Padua, Optimizing data permutations for SIMD devices 41 (6) (2006) 118–131.
- [28] I. Rosen, D. Nuzman, A. Zaks, Loop-aware SLP in GCC, in: *GCC summit, Citeseer*, 2007, pp. 131–142.
- [29] J. Shin, M. Hall, J. Chame, Superword-level parallelism in the presence of control flow, in: *Proceedings of the International Symposium on Code Generation and Optimization*, IEEE Computer Society, 2005, pp. 165–175.
- [30] E. Stupachenko, Programming using avx2. permutations.(2015).
- [31] Y. Sui, X. Fan, H. Zhou, J. Xue, Loop-oriented array- and field-sensitive pointer analysis for automatic SIMD vectorization, in: *Proceedings of the 17th ACM SIGPLAN/SIGBED Conference on Languages, Compilers, Tools, and Theory for Embedded Systems, LCTES 2016, Santa Barbara, CA, USA, June 13–14, 2016*, 2016, pp. 41–51, doi:[10.1145/2907950.2907957](https://doi.org/10.1145/2907950.2907957).
- [32] D. Talla, L.K. John, D. Burger, Bottlenecks in multimedia processing with SIMD style extensions and architectural enhancements, *IEEE Trans. Comput.* 52 (8) (2003) 1015–1031.
- [33] H. Zhou, J. Xue, A compiler approach for exploiting partial SIMD parallelism, *TACO* 13 (1) (2016) 11:1–11:26, doi:[10.1145/2886101](https://doi.org/10.1145/2886101).
- [34] H. Zhou, J. Xue, Exploiting mixed SIMD parallelism by reducing data reorganization overhead, in: *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, ACM, 2016, pp. 59–69.



Sheng-Yu Fu received the M.S. degree in computer science from National Chiao Tung University in 2014. He currently is a Ph.D. candidate in computer science at National Taiwan University. His research interests include dynamic binary translation, virtual machines, SIMD code translation, optimizations for Just-In-Time compilation systems, and vectorization.



Ding-Yong Hong received the M.S. and the Ph.D. degree in computer science from National Tsing Hua University in 2005 and 2013, respectively. He is currently a postdoctoral research fellow and a senior software engineer in the Institute of Information Science, Academia Sinica. His research interests include dynamic binary translation, virtualization, compiler optimization, and parallel and distributed computing.



Yu-Ping Liu received the B.S. and M.S. degree in computer science from National Taiwan University in 2015 and 2017, respectively. He currently is a research assistant at National Taiwan University. His research interests include compiler optimization, just-in-time compilation, and high-performance computing.



Jan-Jan Wu received the B.S. and the M.S. degree in computer science from National Taiwan University in 1985 and 1987 respectively, and the M.S. and Ph.D. in Computer Science from Yale University in 1991 and 1995 respectively. She is currently a research fellow and the leader of the Computer Systems Laboratory in the Institute of Information Science, Academia Sinica. Her current research interests include parallel and distributed computing, cloud computing, cloud databases, virtualization, and dynamic binary translation on multicores.



Wei-Chung Hsu received his PhD degree in computer science from the University of Wisconsin, Madison, in 1987. Currently, he is a professor in the department of computer science and information engineering, at the National Taiwan University, Taipei, Taiwan. From 2009 to 2012, he was a professor at the department of computer science, National Chiao Tung University, Taiwan. From 1999 to 2009, he was a professor at the computer science and engineering department, at University of Minnesota, twin cities. From 1997 to 1999, he was a runtime optimization architect in the California Language Lab, at Hewlett Packard Company. From 1993 to 1997, he was a technical lead in the compiler team which developed and released an optimizing compiler for the HP PA-8000 systems. Prior to joining HP, he was a senior architect at Cray Research, in Chippewa Falls, Wisconsin. His current research interests include high-performance computing systems and architectures, in particular, runtime optimization systems. At Minnesota, he and his students developed the ADORE runtime optimization system that can dynamically optimize programs on-the-fly for the underlying architecture.